

DBGS - PSIS

1.0

Generado por Doxygen 1.13.2

1 DBGS - PSIS	1
2 Instalación y Desarrollo	3
2.1 Proceso de Instalación del Entorno de Desarrollo y la Aplicación	3
2.1.1 Instrucciones de Instalación	3
2.1.1.1 En Linux (Debian/Ubuntu)	3
2.1.1.2 En Windows	3
2.1.2 Proceso de Desarrollo	4
3 Índice de clases	5
3.1 Lista de clases	5
4 Índice de archivos	7
4.1 Lista de archivos	7
5 Documentación de clases	9
5.1 Referencia de la plantilla de la clase BPlusTree< T >	9
5.1.1 Descripción detallada	10
5.1.2 Documentación de constructores y destructores	10
5.1.2.1 BPlusTree()	10
5.1.2.2 ~BPlusTree()	11
5.1.3 Documentación de funciones miembro	11
5.1.3.1 _findIndex()	11
5.1.3.2 _findKey()	11
5.1.3.3 _findRange()	11
5.1.3.4 _innerInsert()	12
5.1.3.5 _insertInChild()	12
5.1.3.6 _insertInParent()	13
5.1.3.7 _insertItem()	13
5.1.3.8 _print()	13
5.1.3.9 _removeFromParent()	14
5.1.3.10 clear()	15
5.1.3.11 getRoot()	15
5.1.3.12 insert()	15
5.1.3.13 remove()	15
5.1.3.14 search()	16
5.1.3.15 searchRange()	16
5.2 Referencia de la estructura Column	17
5.2.1 Descripción detallada	17
5.3 Referencia de la plantilla de la clase HashMap< K, V >	17
5.3.1 Descripción detallada	18
5.3.2 Documentación de constructores y destructores	18
5.3.2.1 HashMap()	18
5.3.2.2 ~HashMap()	19

5.3.3 Documentación de funciones miembro	19
5.3.3.1 _getLoadFactor()	19
5.3.3.2 _getPows()	19
5.3.3.3 _hashCode()	19
5.3.3.4 _insert()	20
5.3.3.5 _toStringGeneric()	20
5.3.3.6 _wrap()	20
5.3.3.7 containsKey()	21
5.3.3.8 get()	21
5.3.3.9 getSize()	21
5.3.3.10 insert()	22
5.3.3.11 isEmpty()	22
5.3.3.12 remove()	22
5.3.3.13 toList()	22
5.4 Referencia de la plantilla de la clase HashNode< K, V >	23
5.4.1 Descripción detallada	23
5.4.2 Documentación de constructores y destructores	23
5.4.2.1 HashNode()	23
5.5 Referencia de la estructura IndexEntry	24
5.5.1 Descripción detallada	24
5.5.2 Documentación de constructores y destructores	24
5.5.2.1 IndexEntry() [1/2]	24
5.5.2.2 IndexEntry() [2/2]	24
5.5.3 Documentación de funciones miembro	25
5.5.3.1 operator<()	25
5.5.3.2 operator<=()	25
5.5.3.3 operator==(())	25
5.5.3.4 operator>()	25
5.5.3.5 operator>=()	26
5.6 Referencia de la clase InvertedIndex	26
5.6.1 Descripción detallada	26
5.6.2 Documentación de constructores y destructores	26
5.6.2.1 InvertedIndex()	26
5.6.2.2 ~InvertedIndex()	27
5.6.3 Documentación de funciones miembro	27
5.6.3.1 add()	27
5.6.3.2 get()	27
5.6.3.3 remove()	27
5.7 Referencia de la plantilla de la estructura Node< T >	28
5.7.1 Descripción detallada	28
5.7.2 Documentación de constructores y destructores	29
5.7.2.1 Node()	29

5.8 Referencia de la estructura <code>ParsedCommand</code>	29
5.8.1 Descripción detallada	30
5.9 Referencia de la clase <code>Parser</code>	30
5.9.1 Descripción detallada	31
5.9.2 Documentación de funciones miembro	31
5.9.2.1 <code>executeCommand()</code>	31
5.9.2.2 <code>executeCreateIndex()</code>	31
5.9.2.3 <code>executeCreateInvertedIndex()</code>	32
5.9.2.4 <code>executeCreateTable()</code>	32
5.9.2.5 <code>executeDelete()</code>	32
5.9.2.6 <code>executeInsert()</code>	33
5.9.2.7 <code>executeSelect()</code>	33
5.9.2.8 <code>executeShowSchema()</code>	33
5.9.2.9 <code>executeShowTables()</code>	34
5.9.2.10 <code>getTable()</code>	34
5.9.2.11 <code>listTables()</code>	34
5.9.2.12 <code>parseCommand()</code>	34
5.9.2.13 <code>parseDataType()</code>	35
5.9.2.14 <code>parseValue()</code>	35
5.9.2.15 <code>split()</code>	35
5.9.2.16 <code>toUpper()</code>	36
5.9.2.17 <code>trim()</code>	36
5.10 Referencia de la clase <code>Table</code>	37
5.10.1 Descripción detallada	38
5.10.2 Documentación de constructores y destructores	38
5.10.2.1 <code>Table()</code>	38
5.10.2.2 <code>~Table()</code>	38
5.10.3 Documentación de funciones miembro	38
5.10.3.1 <code>cellMatchesType()</code>	38
5.10.3.2 <code>columnIndex()</code>	39
5.10.3.3 <code>createBPlusTreeIndex()</code>	39
5.10.3.4 <code>createInvertedIndex()</code>	39
5.10.3.5 <code>dataTypeToString()</code>	39
5.10.3.6 <code>deleteRow()</code>	40
5.10.3.7 <code>getCell()</code>	40
5.10.3.8 <code>getColumnType()</code>	40
5.10.3.9 <code>getRow()</code>	41
5.10.3.10 <code>getSchema()</code>	41
5.10.3.11 <code>insertRow()</code>	41
5.10.3.12 <code>search()</code>	41
5.10.3.13 <code>searchByIndex()</code>	42

6 Documentación de archivos	43
6.1 Referencia del archivo src/Main.cpp	43
6.1.1 Descripción detallada	43
6.1.2 Documentación de funciones	43
6.1.2.1 main()	43
6.2 Referencia del archivo src/structures/BPlusTree.hpp	44
6.2.1 Descripción detallada	44
6.3 BPlusTree.hpp	44
6.4 Referencia del archivo src/structures/HashMap.hpp	53
6.4.1 Descripción detallada	53
6.5 HashMap.hpp	53
6.6 Referencia del archivo src/structures/InvertedIndex.cpp	56
6.6.1 Descripción detallada	56
6.7 InvertedIndex.cpp	57
6.8 Referencia del archivo src/structures/InvertedIndex.hpp	57
6.8.1 Descripción detallada	58
6.9 InvertedIndex.hpp	58
6.10 Referencia del archivo src/system/IndexEntry.cpp	58
6.10.1 Descripción detallada	58
6.11 IndexEntry.cpp	58
6.12 Referencia del archivo src/system/IndexEntry.hpp	59
6.12.1 Descripción detallada	59
6.13 IndexEntry.hpp	60
6.14 Referencia del archivo src/system/Parser.cpp	60
6.14.1 Descripción detallada	60
6.15 Parser.cpp	60
6.16 Referencia del archivo src/system/Parser.hpp	66
6.16.1 Descripción detallada	67
6.16.2 Documentación de enumeraciones	67
6.16.2.1 CommandType	67
6.17 Parser.hpp	67
6.18 Referencia del archivo src/system/Table.cpp	68
6.18.1 Descripción detallada	68
6.19 Table.cpp	69
6.20 Referencia del archivo src/system/Table.hpp	72
6.20.1 Descripción detallada	73
6.21 Table.hpp	73
Índice alfabético	75

Capítulo 1

DBGS - PSIS

DBGS es un sistema gestor de bases de datos extremadamente simple, limitado en funcionalidad y un prototipo experimental. Además que funciona completamente en memoria.

Capítulo 2

Instalación y Desarrollo

2.1. Proceso de Instalación del Entorno de Desarrollo y la Aplicación

Este proyecto requiere la instalación de las siguientes herramientas:

- **GCC**: Compilador de C++.
- **CMake**: Herramienta de construcción multiplataforma.
- **Doxygen**: Generador de documentación.
- **TeX Live**: Distribución de LaTeX para generar documentación en PDF.

2.1.1. Instrucciones de Instalación

2.1.1.1. En Linux (Debian/Ubuntu)

```
sudo apt update
sudo apt install build-essential cmake doxygen texlive-full
```

2.1.1.2. En Windows

1. **GCC**: Instalar [MinGW-w64](#) o [MSYS2](#).
2. **CMake**: Descargar desde [cmake.org](#).
3. **Doxygen**: Descargar desde [doxygen.nl](#).
4. **TeX Live**: Descargar desde [tug.org/texlive](#).

Asegúrese de agregar las herramientas al PATH del sistema.

2.1.2. Proceso de Desarrollo

1. Clonar el repositorio desde <https://github.com/christianmz565/dbgs-psis.git>

2. Realizar las modificaciones necesarias en el código fuente.

3. Para compilar el proyecto, puede ejecutar el script `build.sh`:

```
./build.sh
```

Alternativamente, puede utilizar el archivo `CMakeLists.txt` ubicado en `src/` para configurar y compilar manualmente con CMake.

4. Para regenerar la documentación HTML, ejecute:

```
doxygen Doxyfile
```

5. Si requiere el informe en PDF, diríjase a `docs/latex/` y ejecute:

```
make
```

Esto generará el archivo PDF de la documentación.

Capítulo 3

Índice de clases

3.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

BPlusTree< T >	Implementación genérica de un árbol B+	9
Column	Representa una columna de la tabla, con nombre y tipo de dato	17
HashMap< K, V >	Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento	17
HashNode< K, V >	Nodo que almacena un par clave-valor en la tabla hash	23
IndexEntry	Entrada de índice para estructuras de índice (B+ o invertido)	24
InvertedIndex	Índice invertido basado en HashMap para asociar claves con listas de identificadores	26
Node< T >	Representa un nodo en el árbol B+	28
ParsedCommand	Representa un comando SQL analizado con sus componentes	29
Parser	Analizador SQL simple con soporte de sintaxis de tubería	30
Table	Representa una tabla de datos con soporte para índices y operaciones básicas	37

Capítulo 4

Índice de archivos

4.1. Lista de archivos

Lista de todos los archivos documentados y con breves descripciones:

src/ Main.cpp	
Punto de entrada para la consola interactiva DBGS-PSIS	43
src/structures/ BPlusTree.hpp	
Implementación de un árbol B+ genérico	44
src/structures/ HashMap.hpp	
Implementación de un mapa hash genérico con manejo de colisiones y redimensionamiento automático	53
src/structures/ InvertedIndex.cpp	
Implementación de la clase InvertedIndex	56
src/structures/ InvertedIndex.hpp	
Implementación de un índice invertido utilizando un mapa hash	57
src/system/ IndexEntry.cpp	
Implementación de la estructura IndexEntry para índices de tablas	58
src/system/ IndexEntry.hpp	
Definición de la estructura IndexEntry para índices de tablas	59
src/system/ Parser.cpp	
Implementación del analizador SQL simple con sintaxis de pipas	60
src/system/ Parser.hpp	
Analizador SQL simple con sintaxis de pipas para operaciones de base de datos	66
src/system/ Table.cpp	
Implementación de la clase Table	68
src/system/ Table.hpp	
Definición de la clase Table y estructuras auxiliares para la gestión de tablas de datos	72

Capítulo 5

Documentación de clases

5.1. Referencia de la plantilla de la clase BPlusTree< T >

Implementación genérica de un árbol B+.

```
#include <BPlusTree.hpp>
```

Métodos públicos

- **BPlusTree** (int _degree)
Constructor del árbol B+.
- **~BPlusTree** ()
Destructor del árbol B+.
- **Node**< T > * **_findKey** (Node< T > *node, T key)
Busca el nodo hoja que contiene la clave especificada.
- **Node**< T > * **_findRange** (Node< T > *node, T key)
Busca el nodo hoja donde debería estar la clave.
- int **_findIndex** (T *arr, T data, int len)
Encuentra el índice donde insertar un dato en un arreglo ordenado.
- **Node**< T > ** **_innerInsert** (Node< T > **child_arr, Node< T > *child, int len, int index)
Inserta un hijo en el arreglo de hijos de un nodo interno.
- T * **_insertItem** (T *arr, T data, int len)
Inserta un dato en el arreglo de items de un nodo.
- **Node**< T > * **_insertInChild** (Node< T > *node, T data, Node< T > *child)
Inserta un dato y un hijo en un nodo interno.
- void **_insertInParent** (Node< T > *par, Node< T > *child, T data)
Inserta un nuevo hijo en el nodo padre después de una división.
- void **_removeFromParent** (Node< T > *node, int index, Node< T > *par)
Elimina un hijo del nodo padre.
- void **_print** (Node< T > *cursor)
Imprime recursivamente el árbol desde el nodo dado.
- **Node**< T > * **getRoot** ()
Obtiene la raíz del árbol.
- int **searchRange** (T start, T end, T *result_data, int arr_length)
Busca todos los elementos en el rango [start, end].

- bool **search** (T data)
Busca si un elemento existe en el árbol.
- void **insert** (T data)
Inserta un elemento en el árbol.
- void **remove** (T data)
Elimina un elemento del árbol.
- void **clear** (Node< T > *cursor)
Libera la memoria de todos los nodos del árbol.
- void **print** ()
Imprime el contenido del árbol.

Atributos privados

- Node< T > * **root**
Puntero a la raíz del árbol B+.
- int **degree**
Grado del árbol (máximo número de hijos por nodo).

5.1.1. Descripción detallada

```
template<typename T>
class BPlusTree< T >
```

Implementación genérica de un árbol B+.

Esta clase implementa un árbol B+ que permite almacenar, buscar, eliminar y recorrer elementos de tipo T. El árbol B+ es una estructura de datos balanceada utilizada comúnmente en sistemas de bases de datos y sistemas de archivos.

Parámetros de plantilla

<i>T</i>	Tipo de dato de los elementos almacenados en el árbol.
----------	--

5.1.2. Documentación de constructores y destructores

5.1.2.1. BPlusTree()

```
template<typename T>
BPlusTree< T >::BPlusTree (
    int _degree)
```

Constructor del árbol B+.

Parámetros

<i>_degree</i>	Grado del árbol (máximo número de hijos por nodo).
----------------	--

5.1.2.2. ~BPlusTree()

```
template<typename T>
BPlusTree< T >::~~BPlusTree ()
```

Destructor del árbol B+.

Libera toda la memoria utilizada.

5.1.3. Documentación de funciones miembro**5.1.3.1. _findIndex()**

```
template<typename T>
int BPlusTree< T >::_findIndex (
    T * arr,
    T data,
    int len)
```

Encuentra el índice donde insertar un dato en un arreglo ordenado.

Parámetros

<i>arr</i>	Arreglo de datos.
<i>data</i>	Dato a insertar.
<i>len</i>	Longitud del arreglo.

Devuelve

Índice de inserción.

5.1.3.2. _findKey()

```
template<typename T>
Node< T > * BPlusTree< T >::_findKey (
    Node< T > * node,
    T key)
```

Busca el nodo hoja que contiene la clave especificada.

Parámetros

<i>node</i>	Nodo desde el cual iniciar la búsqueda.
<i>key</i>	Clave a buscar.

Devuelve

Puntero al nodo que contiene la clave, o nullptr si no se encuentra.

5.1.3.3. _findRange()

```
template<typename T>
Node< T > * BPlusTree< T >::_findRange (
    Node< T > * node,
    T key)
```

Busca el nodo hoja donde debería estar la clave.

Parámetros

<i>node</i>	Nodo desde el cual iniciar la búsqueda.
<i>key</i>	Clave a buscar.

Devuelve

Puntero al nodo hoja correspondiente.

5.1.3.4. _innerInsert()

```
template<typename T>
Node< T > ** BPlusTree< T >::_innerInsert (
    Node< T > ** child_arr,
    Node< T > * child,
    int len,
    int index)
```

Inserta un hijo en el arreglo de hijos de un nodo interno.

Parámetros

<i>child_arr</i>	Arreglo de hijos.
<i>child</i>	Nuevo hijo a insertar.
<i>len</i>	Longitud actual del arreglo.
<i>index</i>	Índice donde insertar el nuevo hijo.

Devuelve

Arreglo de hijos actualizado.

5.1.3.5. _insertInChild()

```
template<typename T>
Node< T > * BPlusTree< T >::_insertInChild (
    Node< T > * node,
    T data,
    Node< T > * child)
```

Inserta un dato y un hijo en un nodo interno.

Parámetros

<i>node</i>	Nodo interno.
<i>data</i>	Dato a insertar.
<i>child</i>	Hijo a insertar.

Devuelve

Nodo actualizado.

5.1.3.6. _insertInParent()

```
template<typename T>
void BPlusTree< T >::_insertInParent (
    Node< T > * par,
    Node< T > * child,
    T data)
```

Inserta un nuevo hijo en el nodo padre después de una división.

Parámetros

<i>par</i>	Nodo padre.
<i>child</i>	Nuevo hijo.
<i>data</i>	Dato promovido.

5.1.3.7. _insertItem()

```
template<typename T>
T * BPlusTree< T >::_insertItem (
    T * arr,
    T data,
    int len)
```

Inserta un dato en el arreglo de items de un nodo.

Parámetros

<i>arr</i>	Arreglo de items.
<i>data</i>	Dato a insertar.
<i>len</i>	Longitud actual del arreglo.

Devuelve

Arreglo de items actualizado.

5.1.3.8. _print()

```
template<typename T>
void BPlusTree< T >::_print (
    Node< T > * cursor)
```

Imprime recursivamente el árbol desde el nodo dado.

Parámetros

<i>cursor</i>	Nodo desde el cual imprimir.
---------------	------------------------------

5.1.3.9. `_removeFromParent()`

```
template<typename T>
void BPlusTree< T >::_removeFromParent (
    Node< T > * node,
    int index,
    Node< T > * par)
```

Elimina un hijo del nodo padre.

Parámetros

<i>node</i>	Nodo a eliminar.
<i>index</i>	Índice del hijo a eliminar.
<i>par</i>	Nodo padre.

5.1.3.10. clear()

```
template<typename T>
void BPlusTree< T >::clear (
    Node< T > * cursor)
```

Libera la memoria de todos los nodos del árbol.

Parámetros

<i>cursor</i>	Nodo desde el cual iniciar la liberación.
---------------	---

5.1.3.11. getRoot()

```
template<typename T>
Node< T > * BPlusTree< T >::getRoot ()
```

Obtiene la raíz del árbol.

Devuelve

Puntero a la raíz.

5.1.3.12. insert()

```
template<typename T>
void BPlusTree< T >::insert (
    T data)
```

Inserta un elemento en el árbol.

Parámetros

<i>data</i>	Elemento a insertar.
-------------	----------------------

5.1.3.13. remove()

```
template<typename T>
void BPlusTree< T >::remove (
    T data)
```

Elimina un elemento del árbol.

Parámetros

<i>data</i>	Elemento a eliminar.
-------------	----------------------

5.1.3.14. search()

```
template<typename T>
bool BPlusTree< T >::search (
    T data)
```

Busca si un elemento existe en el árbol.

Parámetros

<i>data</i>	Elemento a buscar.
-------------	--------------------

Devuelve

true si existe, false en caso contrario.

5.1.3.15. searchRange()

```
template<typename T>
int BPlusTree< T >::searchRange (
    T start,
    T end,
    T * result_data,
    int arr_length)
```

Busca todos los elementos en el rango [start, end].

Parámetros

<i>start</i>	Límite inferior del rango.
<i>end</i>	Límite superior del rango.
<i>result_data</i>	Arreglo donde se almacenan los resultados.
<i>arr_length</i>	Longitud máxima del arreglo de resultados.

Devuelve

Número de elementos encontrados.

La documentación de esta clase está generada del siguiente archivo:

- [src/structures/BPlusTree.hpp](#)

5.2. Referencia de la estructura Column

Representa una columna de la tabla, con nombre y tipo de dato.

```
#include <Table.hpp>
```

Atributos públicos

- `std::string name`
Nombre de la columna.
- `DataType type`
Tipo de dato de la columna.

5.2.1. Descripción detallada

Representa una columna de la tabla, con nombre y tipo de dato.

La documentación de esta estructura está generada del siguiente archivo:

- `src/system/`[Table.hpp](#)

5.3. Referencia de la plantilla de la clase `HashMap< K, V >`

Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento.

```
#include <HashMap.hpp>
```

Métodos públicos

- `HashMap (int cap)`
Constructor de [HashMap](#).
- `~HashMap ()`
Destructor de [HashMap](#).
- `long long * _getPows ()`
Calcula y retorna el arreglo de potencias para el hash.
- `double _getLoadFactor ()`
Calcula el factor de carga actual de la tabla.
- `std::string _toStringGeneric (const K &val)`
Convierte un valor genérico a string para el hash.
- `int _hashCode (K key)`
Calcula el código hash para una clave.
- `int _wrap (long long key, int size)`
Ajusta un valor hash al rango de la tabla.
- `void _insert (K key, V value, HashNode< K, V > **arr)`
Inserta un par clave-valor en un arreglo de nodos.
- `void _grow ()`
Duplica la capacidad de la tabla y reubica los elementos.

- void **insert** (K key, V value)
Inserta un par clave-valor en el mapa.
- std::optional< V > **get** (K key)
Obtiene el valor asociado a una clave.
- std::optional< V > **remove** (K key)
Elimina un par clave-valor del mapa.
- bool **containsKey** (K key)
Verifica si una clave existe en el mapa.
- int **getSize** ()
Retorna el número de elementos almacenados.
- bool **isEmpty** ()
Verifica si el mapa está vacío.
- **HashNode**< K, V > ** **toList** ()
Retorna un arreglo con todos los nodos almacenados.
- void **display** ()
Muestra por consola todos los pares clave-valor almacenados.
- void **clear** ()
Elimina todos los elementos del mapa y libera la memoria.

Atributos privados

- **HashNode**< K, V > ** **table**
Tabla hash de punteros a nodos.
- int **capacity**
Capacidad actual de la tabla.
- int **size**
Número de elementos almacenados.
- long long * **pows**
Potencias precomputadas para el hash.

5.3.1. Descripción detallada

template<typename K, typename V>
class **HashMap**< K, V >

Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento.

Permite almacenar, buscar, eliminar y mostrar pares clave-valor. Utiliza direccionamiento abierto con sondeo cuadrático para resolver colisiones y redimensiona automáticamente la tabla cuando la carga supera cierto umbral.

Parámetros de plantilla

<i>K</i>	Tipo de la clave.
<i>V</i>	Tipo del valor.

5.3.2. Documentación de constructores y destructores

5.3.2.1. HashMap()

```
template<typename K, typename V>
HashMap< K, V >::HashMap (
    int cap)
```

Constructor de **HashMap**.

Parámetros

<code>cap</code>	Capacidad inicial de la tabla hash.
------------------	-------------------------------------

Nota

La capacidad debe ser un número potencia de 2 para evitar escenarios donde el redireccionamiento no encuentre un espacio libre. Se recomienda usar un valor mayor a 16 para evitar colisiones frecuentes.

Parámetros de plantilla

<code>K</code>	Tipo de la clave.
<code>V</code>	Tipo del valor.

5.3.2.2. `~HashMap()`

```
template<typename K, typename V>
HashMap< K, V >::~~HashMap ()
```

Destructor de `HashMap`.

Libera la memoria utilizada.

5.3.3. Documentación de funciones miembro**5.3.3.1. `_getLoadFactor()`**

```
template<typename K, typename V>
double HashMap< K, V >::_getLoadFactor ()
```

Calcula el factor de carga actual de la tabla.

Devuelve

Factor de carga (size/capacity).

5.3.3.2. `_getPows()`

```
template<typename K, typename V>
long long * HashMap< K, V >::_getPows ()
```

Calcula y retorna el arreglo de potencias para el hash.

Devuelve

Puntero al arreglo de potencias.

5.3.3.3. `_hashCode()`

```
template<typename K, typename V>
int HashMap< K, V >::_hashCode (
    K key)
```

Calcula el código hash para una clave.

Parámetros

<i>key</i>	Clave a hashear.
------------	------------------

Devuelve

Índice hash calculado.

5.3.3.4. `_insert()`

```
template<typename K, typename V>
void HashMap< K, V >::_insert (
    K key,
    V value,
    HashNode< K, V > ** arr)
```

Inserta un par clave-valor en un arreglo de nodos.

Parámetros

<i>key</i>	Clave a insertar.
<i>value</i>	Valor a insertar.
<i>arr</i>	Arreglo de nodos donde insertar.

5.3.3.5. `_toStringGeneric()`

```
template<typename K, typename V>
std::string HashMap< K, V >::_toStringGeneric (
    const K & val)
```

Convierte un valor genérico a string para el hash.

Parámetros

<i>val</i>	Valor a convertir.
------------	--------------------

Devuelve

Representación en string del valor.

5.3.3.6. `_wrap()`

```
template<typename K, typename V>
int HashMap< K, V >::_wrap (
    long long key,
    int size)
```

Ajusta un valor hash al rango de la tabla.

Parámetros

<i>key</i>	Valor hash.
<i>size</i>	Tamaño de la tabla.

Devuelve

Valor ajustado al rango [0, size).

5.3.3.7. `containsKey()`

```
template<typename K, typename V>
bool HashMap< K, V >::containsKey (
    K key)
```

Verifica si una clave existe en el mapa.

Parámetros

<i>key</i>	Clave a buscar.
------------	-----------------

Devuelve

true si existe, false en caso contrario.

5.3.3.8. `get()`

```
template<typename K, typename V>
std::optional< V > HashMap< K, V >::get (
    K key)
```

Obtiene el valor asociado a una clave.

Parámetros

<i>key</i>	Clave a buscar.
------------	-----------------

Devuelve

Valor asociado si existe, `std::nullopt` en caso contrario.

5.3.3.9. `getSize()`

```
template<typename K, typename V>
int HashMap< K, V >::getSize ()
```

Retorna el número de elementos almacenados.

Devuelve

Número de elementos.

5.3.3.10. insert()

```
template<typename K, typename V>
void HashMap< K, V >::insert (
    K key,
    V value)
```

Inserta un par clave-valor en el mapa.

Parámetros

<i>key</i>	Clave a insertar.
<i>value</i>	Valor a insertar.

5.3.3.11. isEmpty()

```
template<typename K, typename V>
bool HashMap< K, V >::isEmpty ()
```

Verifica si el mapa está vacío.

Devuelve

true si está vacío, false en caso contrario.

5.3.3.12. remove()

```
template<typename K, typename V>
std::optional< V > HashMap< K, V >::remove (
    K key)
```

Elimina un par clave-valor del mapa.

Parámetros

<i>key</i>	Clave a eliminar.
------------	-------------------

Devuelve

Valor eliminado si existía, std::nullopt en caso contrario.

5.3.3.13. toList()

```
template<typename K, typename V>
HashNode< K, V > ** HashMap< K, V >::toList ()
```

Retorna un arreglo con todos los nodos almacenados.

Devuelve

Puntero al arreglo de nodos.

La documentación de esta clase está generada del siguiente archivo:

- [src/structures/HashMap.hpp](#)

5.4. Referencia de la plantilla de la clase `HashNode< K, V >`

Nodo que almacena un par clave-valor en la tabla hash.

```
#include <HashMap.hpp>
```

Métodos públicos

- `HashNode` (`K key`, `V value`)
Constructor de `HashNode`.

Atributos públicos

- `V value`
Valor almacenado en el nodo.
- `K key`
Clave asociada al valor.

5.4.1. Descripción detallada

```
template<typename K, typename V>
class HashNode< K, V >
```

Nodo que almacena un par clave-valor en la tabla hash.

Parámetros de plantilla

<code>K</code>	Tipo de la clave.
<code>V</code>	Tipo del valor.

5.4.2. Documentación de constructores y destructores

5.4.2.1. `HashNode()`

```
template<typename K, typename V>
HashNode< K, V >::HashNode (
    K key,
    V value)
```

Constructor de `HashNode`.

Parámetros

<code>key</code>	Clave del nodo.
<code>value</code>	Valor asociado a la clave.

La documentación de esta clase está generada del siguiente archivo:

- `src/structures/HashMap.hpp`

5.5. Referencia de la estructura IndexEntry

Entrada de índice para estructuras de índice (B+ o invertido).

```
#include <IndexEntry.hpp>
```

Métodos públicos

- `IndexEntry ()`
Constructor por defecto.
- `IndexEntry (const Cell &k, int r)`
Constructor con parámetros.
- `bool operator< (const IndexEntry &other) const`
Operador de comparación menor que.
- `bool operator> (const IndexEntry &other) const`
Operador de comparación mayor que.
- `bool operator<= (const IndexEntry &other) const`
Operador de comparación menor o igual que.
- `bool operator>= (const IndexEntry &other) const`
Operador de comparación mayor o igual que.
- `bool operator== (const IndexEntry &other) const`
Operador de comparación de igualdad.

Atributos públicos

- `Cell key`
Clave del índice (valor de celda).
- `int rowId`
Identificador de la fila.

5.5.1. Descripción detallada

Entrada de índice para estructuras de índice (B+ o invertido).

Contiene la clave (valor de celda) y el identificador de fila correspondiente.

5.5.2. Documentación de constructores y destructores

5.5.2.1. IndexEntry() [1/2]

```
IndexEntry::IndexEntry () [inline]
```

Constructor por defecto.

Crea una entrada de índice vacía con valores por defecto.

5.5.2.2. IndexEntry() [2/2]

```
IndexEntry::IndexEntry (
    const Cell & k,
    int r) [inline]
```

Constructor con parámetros.

Parámetros

<i>k</i>	Clave del índice
<i>r</i>	Identificador de fila

5.5.3. Documentación de funciones miembro

5.5.3.1. operator<()

```
bool IndexEntry::operator< (  
    const IndexEntry & other) const
```

Operador de comparación menor que.

Compara dos [IndexEntry](#) basándose en su clave.

5.5.3.2. operator<=()

```
bool IndexEntry::operator<= (  
    const IndexEntry & other) const
```

Operador de comparación menor o igual que.

Compara dos [IndexEntry](#) basándose en su clave.

5.5.3.3. operator==(())

```
bool IndexEntry::operator==(   
    const IndexEntry & other) const
```

Operador de comparación de igualdad.

Compara dos [IndexEntry](#) basándose en su clave y rowId.

5.5.3.4. operator>()

```
bool IndexEntry::operator> (  
    const IndexEntry & other) const
```

Operador de comparación mayor que.

Compara dos [IndexEntry](#) basándose en su clave.

5.5.3.5. operator>=()

```
bool IndexEntry::operator>= (
    const IndexEntry & other) const
```

Operador de comparación mayor o igual que.

Compara dos [IndexEntry](#) basándose en su clave.

La documentación de esta estructura está generada de los siguientes archivos:

- [src/system/IndexEntry.hpp](#)
- [src/system/IndexEntry.cpp](#)

5.6. Referencia de la clase InvertedIndex

Índice invertido basado en [HashMap](#) para asociar claves con listas de identificadores.

```
#include <InvertedIndex.hpp>
```

Métodos públicos

- [InvertedIndex](#) (int initialCapacity=128)
Constructor de [InvertedIndex](#).
- [~InvertedIndex](#) ()
Destructor de [InvertedIndex](#).
- void [add](#) (const std::string &key, int rowId)
Agrega una asociación entre una clave y un identificador de fila.
- std::vector< int > [get](#) (const std::string &key)
Obtiene la lista de identificadores asociados a una clave.
- void [remove](#) (const std::string &key, int rowId)
Elimina la asociación de un identificador de fila para una clave dada.
- void [clear](#) ()
Elimina todas las asociaciones del índice invertido.

Atributos privados

- [HashMap](#)< std::string, std::vector< int > > [map_](#)
Mapa hash que almacena las asociaciones clave-lista de identificadores.

5.6.1. Descripción detallada

Índice invertido basado en [HashMap](#) para asociar claves con listas de identificadores.

Permite agregar, obtener, eliminar y limpiar asociaciones entre claves y listas de enteros.

5.6.2. Documentación de constructores y destructores

5.6.2.1. InvertedIndex()

```
InvertedIndex::InvertedIndex (
    int initialCapacity = 128)
```

Constructor de [InvertedIndex](#).

Parámetros

<i>initialCapacity</i>	Capacidad inicial del mapa hash subyacente (por defecto 128).
------------------------	---

5.6.2.2. `~InvertedIndex()`

```
InvertedIndex::~~InvertedIndex ()
```

Destructor de [InvertedIndex](#).

Libera la memoria utilizada.

5.6.3. Documentación de funciones miembro

5.6.3.1. `add()`

```
void InvertedIndex::add (
    const std::string & key,
    int rowId)
```

Agrega una asociación entre una clave y un identificador de fila.

Parámetros

<i>key</i>	Clave a asociar.
<i>rowId</i>	Identificador de fila a agregar.

5.6.3.2. `get()`

```
std::vector< int > InvertedIndex::get (
    const std::string & key)
```

Obtiene la lista de identificadores asociados a una clave.

Parámetros

<i>key</i>	Clave a buscar.
------------	-----------------

Devuelve

Vector de identificadores asociados. Si la clave no existe, retorna un vector vacío.

5.6.3.3. `remove()`

```
void InvertedIndex::remove (
    const std::string & key,
    int rowId)
```

Elimina la asociación de un identificador de fila para una clave dada.

Parámetros

<i>key</i>	Clave de la que se eliminará el identificador.
<i>row↔ ld</i>	Identificador de fila a eliminar.

La documentación de esta clase está generada de los siguientes archivos:

- [src/structures/InvertedIndex.hpp](#)
- [src/structures/InvertedIndex.cpp](#)

5.7. Referencia de la plantilla de la estructura Node< T >

Representa un nodo en el árbol B+.

```
#include <BPlusTree.hpp>
```

Métodos públicos

- [Node](#) (int *_degree*)
Constructor del nodo.

Atributos públicos

- bool **is_leaf**
Indica si el nodo es una hoja.
- int **degree**
Grado del nodo, que determina cuántos hijos puede tener.
- int **size**
Número actual de elementos en el nodo.
- T * **item**
Arreglo de elementos almacenados en el nodo.
- [Node](#)< T > ** **children**
Arreglo de punteros a los hijos del nodo.
- [Node](#)< T > * **parent**
Puntero al nodo padre.

5.7.1. Descripción detallada

```
template<typename T>
struct Node< T >
```

Representa un nodo en el árbol B+.

Cada nodo puede ser una hoja o un nodo interno. Los nodos internos contienen claves y punteros a sus hijos, mientras que las hojas contienen los datos finales del árbol.

5.7.2. Documentación de constructores y destructores

5.7.2.1. Node()

```
template<typename T>
Node< T >::Node (
    int _degree)
```

Constructor del nodo.

Constructor de un nodo del árbol B+.

Parámetros

<code>_degree</code>	Grado del nodo, que determina cuántos hijos puede tener.
----------------------	--

Inicializa un nodo con el grado especificado. El nodo es conectado al árbol B+ en la función de inserción.

Parámetros

<code>_degree</code>	El grado del nodo, que determina cuántos hijos puede tener este nodo.
----------------------	---

La documentación de esta estructura está generada del siguiente archivo:

- [src/structures/BPlusTree.hpp](#)

5.8. Referencia de la estructura ParsedCommand

Representa un comando SQL analizado con sus componentes.

```
#include <Parser.hpp>
```

Atributos públicos

- **CommandType** type
Tipo de comando.
- std::string **tableName**
Nombre de la tabla.
- std::vector< std::string > **columns**
Columnas involucradas.
- std::vector< std::string > **values**
Valores a insertar o comparar.
- std::string **whereColumn**
Columna para condición WHERE.
- std::string **whereValue**
Valor para condición WHERE.
- std::string **indexColumn**
Columna para índice.
- int **indexDegree** = 3
Grado por defecto del B+ tree.

5.8.1. Descripción detallada

Representa un comando SQL analizado con sus componentes.

La documentación de esta estructura está generada del siguiente archivo:

- `src/system/Parser.hpp`

5.9. Referencia de la clase Parser

Analizador SQL simple con soporte de sintaxis de tubería.

```
#include <Parser.hpp>
```

Métodos públicos

- **Parser ()**
Constructor de [Parser](#).
- **~Parser ()**
Destructor de [Parser](#).
- **bool executeCommand** (const std::string &command)
Analiza y ejecuta un comando SQL.
- **ParsedCommand parseCommand** (const std::string &command)
Analiza una cadena de comando en una estructura [ParsedCommand](#).
- **Table * getTable** (const std::string &tableName)
Obtiene una referencia a una tabla por nombre.
- **std::vector< std::string > listTables () const**
Lista todas las tablas disponibles.

Métodos privados

- **bool executeCreateTable** (const [ParsedCommand](#) &cmd)
Ejecuta un comando CREATE TABLE.
- **bool executeInsert** (const [ParsedCommand](#) &cmd)
Ejecuta un comando INSERT.
- **bool executeSelect** (const [ParsedCommand](#) &cmd)
Ejecuta un comando SELECT.
- **bool executeDelete** (const [ParsedCommand](#) &cmd)
Ejecuta un comando DELETE.
- **bool executeCreateIndex** (const [ParsedCommand](#) &cmd)
Ejecuta un comando CREATE INDEX.
- **bool executeCreateInvertedIndex** (const [ParsedCommand](#) &cmd)
Ejecuta un comando CREATE INVERTED INDEX.
- **bool executeShowSchema** (const [ParsedCommand](#) &cmd)
Ejecuta un comando SHOW SCHEMA.
- **bool executeShowTables ()**
Ejecuta un comando SHOW TABLES.
- **DataTypes parseDataType** (const std::string &typeStr)

Parsea una cadena de tipo de dato a *DataType*.

- `Cell parseValue` (const std::string &valueStr, *DataType* dataType)

Parsea una cadena de valor al tipo *Cell* correspondiente.

- `std::vector< std::string > split` (const std::string &str, char delimiter, char escapeChar='\0')

Divide una cadena por un delimitador.

- `std::string trim` (const std::string &str)

Elimina espacios en blanco de una cadena.

- `std::string toUpper` (const std::string &str)

Convierte una cadena a mayúsculas.

Atributos privados

- `std::unordered_map< std::string, std::unique_ptr< Table > > tables_`

5.9.1. Descripción detallada

Analizador SQL simple con soporte de sintaxis de tubería.

Analiza y ejecuta comandos tipo SQL usando una sintaxis separada por pipas. Ejemplo: "CREATE TABLE users | name:STRING age:INT | INSERT values"

5.9.2. Documentación de funciones miembro

5.9.2.1. executeCommand()

```
bool Parser::executeCommand (
    const std::string & command)
```

Analiza y ejecuta un comando SQL.

Parámetros

<i>command</i>	Cadena del comando SQL a analizar y ejecutar.
----------------	---

Devuelve

true si el comando se ejecutó correctamente, false en caso contrario.

5.9.2.2. executeCreateIndex()

```
bool Parser::executeCreateIndex (
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando CREATE INDEX.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.3. executeCreateInvertedIndex()

```
bool Parser::executeCreateInvertedIndex (  
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando CREATE INVERTED INDEX.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.4. executeCreateTable()

```
bool Parser::executeCreateTable (  
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando CREATE TABLE.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.5. executeDelete()

```
bool Parser::executeDelete (  
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando DELETE.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.6. executeInsert()

```
bool Parser::executeInsert (  
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando INSERT.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.7. executeSelect()

```
bool Parser::executeSelect (  
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando SELECT.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.8. executeShowSchema()

```
bool Parser::executeShowSchema (  
    const ParsedCommand & cmd) [private]
```

Ejecuta un comando SHOW SCHEMA.

Parámetros

<i>cmd</i>	Estructura de comando analizado.
------------	----------------------------------

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.9. executeShowTables()

```
bool Parser::executeShowTables () [private]
```

Ejecuta un comando SHOW TABLES.

Devuelve

true si fue exitoso, false en caso contrario.

5.9.2.10. getTable()

```
Table * Parser::getTable (  
    const std::string & tableName)
```

Obtiene una referencia a una tabla por nombre.

Parámetros

<i>tableName</i>	Nombre de la tabla a recuperar.
------------------	---------------------------------

Devuelve

Puntero a la tabla, o nullptr si no se encuentra.

5.9.2.11. listTables()

```
std::vector< std::string > Parser::listTables () const
```

Lista todas las tablas disponibles.

Devuelve

Vector con los nombres de las tablas.

5.9.2.12. parseCommand()

```
ParsedCommand Parser::parseCommand (  
    const std::string & command)
```

Analiza una cadena de comando en una estructura [ParsedCommand](#).

Parámetros

<i>command</i>	Cadena del comando a analizar.
----------------	--------------------------------

Devuelve

Estructura [ParsedCommand](#) con los componentes analizados.

5.9.2.13. parseDataType()

```
DataType Parser::parseDataType (  
    const std::string & typeStr) [private]
```

Parsea una cadena de tipo de dato a [DataType](#).

Parámetros

<i>typeStr</i>	Cadena con la representación del tipo de dato.
----------------	--

Devuelve

Valor del enum [DataType](#).

5.9.2.14. parseValue()

```
Cell Parser::parseValue (  
    const std::string & valueStr,  
    DataType dataType) [private]
```

Parsea una cadena de valor al tipo [Cell](#) correspondiente.

Parámetros

<i>valueStr</i>	Cadena con el valor.
<i>dataType</i>	Tipo de dato esperado.

Devuelve

[Cell](#) con el valor parseado.

5.9.2.15. split()

```
std::vector< std::string > Parser::split (  
    const std::string & str,  
    char delimiter,  
    char escapeChar = '\\0') [private]
```

Divide una cadena por un delimitador.

Parámetros

<i>str</i>	Cadena a dividir.
<i>delimiter</i>	Carácter delimitador.
<i>escapeChar</i>	Carácter de escape.

Devuelve

Vector de cadenas resultantes.

5.9.2.16. toUpper()

```
std::string Parser::toUpper (
    const std::string & str) [private]
```

Convierte una cadena a mayúsculas.

Parámetros

<i>str</i>	Cadena a convertir.
------------	---------------------

Devuelve

Cadena en mayúsculas.

5.9.2.17. trim()

```
std::string Parser::trim (
    const std::string & str) [private]
```

Elimina espacios en blanco de una cadena.

Parámetros

<i>str</i>	Cadena a limpiar.
------------	-------------------

Devuelve

Cadena sin espacios en blanco.

La documentación de esta clase está generada de los siguientes archivos:

- [src/system/Parser.hpp](#)
- [src/system/Parser.cpp](#)

5.10. Referencia de la clase Table

Representa una tabla de datos con soporte para índices y operaciones básicas.

```
#include <Table.hpp>
```

Métodos públicos

- **Table** (const std::vector< std::pair< std::string, **DataType** > > &cols, int hashCapacity=128)
Constructor de Table.
- **~Table** ()
Destructor de Table.
- void **insertRow** (const std::vector< Cell > &row)
Inserta una nueva fila en la tabla.
- **DataType** & **getColumnType** (const std::string &colName)
Obtiene el tipo de dato de una columna dado su nombre.
- void **printSchema** () const
Imprime el esquema (columnas y tipos) de la tabla.
- void **printAllRows** () const
Imprime todas las filas almacenadas en la tabla.
- Cell & **getCell** (int rowIndex, const std::string &colName)
Obtiene el valor de una celda dado el índice de fila y el nombre de columna.
- std::vector< Cell > & **getRow** (int rowIndex)
Obtiene una fila completa dado su índice.
- std::vector< **DataType** > **getSchema** () const
Obtiene el esquema de la tabla como un vector de tipos de datos.
- std::vector< Cell > & **deleteRow** (int rowIndex)
Elimina una fila dada su índice.
- void **createBPlusTreeIndex** (const std::string &colName, int degree)
Crea un índice B+ sobre una columna específica.
- void **createInvertedIndex** (const std::string &colName)
Crea un índice invertido sobre una columna específica.
- std::vector< int > **searchByIndex** (const std::string &colName, const Cell &key)
Busca filas por valor de clave usando el índice correspondiente.
- std::vector< int > **search** (const std::string &colName, const Cell &key)
Función de búsqueda general, si la columna tiene un índice, lo usa; de lo contrario, realiza una búsqueda lineal.

Métodos privados

- int **columnIndex** (const std::string &colName)
Obtiene el índice de una columna dado su nombre.

Métodos privados estáticos

- static bool **cellMatchesType** (const Cell &cell, **DataType** dt)
Verifica si una celda coincide con un tipo de dato.
- static std::string **dataTypeToString** (**DataType** dt)
Convierte un tipo de dato a su representación en string.

Atributos privados

- `std::vector< Column > schema_`
Esquema de columnas de la tabla.
- `HashMap< std::string, int > nameToIndex_`
Mapa de nombre de columna a índice.
- `std::vector< std::vector< Cell > > data_`
Datos almacenados en la tabla.
- `HashMap< std::string, BPlusTree< IndexEntry > * > bptIndices_`
Índices B+ por columna.
- `HashMap< std::string, InvertedIndex * > invIndices_`
Índices invertidos por columna.

5.10.1. Descripción detallada

Representa una tabla de datos con soporte para índices y operaciones básicas.

Permite definir un esquema de columnas, insertar filas, crear índices B+ e invertidos, y realizar búsquedas eficientes por índice.

5.10.2. Documentación de constructores y destructores

5.10.2.1. Table()

```
Table::Table (
    const std::vector< std::pair< std::string, DataType > > & cols,
    int hashCapacity = 128)
```

Constructor de [Table](#).

Parámetros

<i>cols</i>	Vector de pares (nombre, tipo) que define el esquema de la tabla.
<i>hashCapacity</i>	Capacidad inicial para los mapas hash internos (por defecto 128).

5.10.2.2. ~Table()

```
Table::~~Table ()
```

Destructor de [Table](#).

Libera la memoria utilizada por los índices.

5.10.3. Documentación de funciones miembro

5.10.3.1. cellMatchesType()

```
bool Table::cellMatchesType (
    const Cell & cell,
    DataType dt) [static], [private]
```

Verifica si una celda coincide con un tipo de dato.

Parámetros

<i>cell</i>	Celda a verificar.
<i>dt</i>	Tipo de dato esperado.

Devuelve

true si coincide, false en caso contrario.

5.10.3.2. columnIndex()

```
int Table::columnIndex (  
    const std::string & colName) [private]
```

Obtiene el índice de una columna dado su nombre.

Parámetros

<i>colName</i>	Nombre de la columna.
----------------	-----------------------

Devuelve

Índice de la columna en el esquema.

5.10.3.3. createBPlusTreeIndex()

```
void Table::createBPlusTreeIndex (  
    const std::string & colName,  
    int degree)
```

Crea un índice B+ sobre una columna específica.

Parámetros

<i>colName</i>	Nombre de la columna a indexar.
<i>degree</i>	Grado del árbol B+.

5.10.3.4. createInvertedIndex()

```
void Table::createInvertedIndex (  
    const std::string & colName)
```

Crea un índice invertido sobre una columna específica.

Parámetros

<i>colName</i>	Nombre de la columna a indexar.
----------------	---------------------------------

5.10.3.5. dataTypeToString()

```
std::string Table::dataTypeToString (  
    DataType dt) [static], [private]
```

Convierte un tipo de dato a su representación en string.

Parámetros

<i>dt</i>	Tipo de dato.
-----------	---------------

Devuelve

String representando el tipo de dato.

5.10.3.6. deleteRow()

```
std::vector< Cell > & Table::deleteRow (  
    int rowIndex)
```

Elimina una fila dada su índice.

Parámetros

<i>rowIndex</i>	Índice de la fila a eliminar.
-----------------	-------------------------------

Devuelve

Vector de celdas que representaba la fila eliminada.

5.10.3.7. getCell()

```
Cell & Table::getCell (  
    int rowIndex,  
    const std::string & colName)
```

Obtiene el valor de una celda dado el índice de fila y el nombre de columna.

Parámetros

<i>rowIndex</i>	Índice de la fila.
<i>colName</i>	Nombre de la columna.

Devuelve

Valor de la celda correspondiente.

5.10.3.8. getColumnType()

```
DataType & Table::getColumnType (  
    const std::string & colName)
```

Obtiene el tipo de dato de una columna dado su nombre.

Parámetros

<i>colName</i>	Nombre de la columna.
----------------	-----------------------

Devuelve

Tipo de dato de la columna.

5.10.3.9. `getRow()`

```
std::vector< Cell > & Table::getRow (  
    int rowIndex)
```

Obtiene una fila completa dado su índice.

Parámetros

<i>rowIndex</i>	Índice de la fila.
-----------------	--------------------

Devuelve

Vector de celdas que representa la fila.

5.10.3.10. `getSchema()`

```
std::vector< DataType > Table::getSchema () const
```

Obtiene el esquema de la tabla como un vector de tipos de datos.

Devuelve

Vector de [DataType](#) representando el esquema.

5.10.3.11. `insertRow()`

```
void Table::insertRow (  
    const std::vector< Cell > & row)
```

Inserta una nueva fila en la tabla.

Parámetros

<i>row</i>	Vector de celdas que representa la fila a insertar.
------------	---

5.10.3.12. `search()`

```
std::vector< int > Table::search (  
    const std::string & colName,  
    const Cell & key)
```

Función de búsqueda general, si la columna tiene un índice, lo usa; de lo contrario, realiza una búsqueda lineal.

Parámetros

<i>colName</i>	El nombre de la columna a buscar.
<i>key</i>	El valor a buscar.

Devuelve

Vector de índices de fila donde la columna coincide con la clave.

5.10.3.13. searchByIndex()

```
std::vector< int > Table::searchByIndex (
    const std::string & colName,
    const Cell & key)
```

Busca filas por valor de clave usando el índice correspondiente.

Parámetros

<i>colName</i>	Nombre de la columna indexada.
<i>key</i>	Valor de la clave a buscar.

Devuelve

Vector de identificadores de filas que coinciden con la clave.

La documentación de esta clase está generada de los siguientes archivos:

- [src/system/Table.hpp](#)
- [src/system/Table.cpp](#)

Capítulo 6

Documentación de archivos

6.1. Referencia del archivo src/Main.cpp

Punto de entrada para la consola interactiva DBGS-PSIS.

```
#include "system/Parser.cpp"  
#include <iostream>  
#include <string>
```

Funciones

- `int main ()`

Punto de entrada principal del programa.

6.1.1. Descripción detallada

Punto de entrada para la consola interactiva DBGS-PSIS.

Este programa lanza una interfaz similar a un terminal que permite a los usuarios ingresar y ejecutar comandos de base de datos de forma interactiva. Escriba 'exit' o 'quit' para salir.

6.1.2. Documentación de funciones

6.1.2.1. main()

```
int main ()
```

Punto de entrada principal del programa.

Inicia la consola interactiva y permite a los usuarios ingresar comandos. El programa continuará ejecutándose hasta que el usuario escriba 'exit' o 'quit'.

6.2. Referencia del archivo src/structures/BPlusTree.hpp

Implementación de un árbol B+ genérico.

```
#include <iostream>
```

Clases

- struct `Node< T >`
Representa un nodo en el árbol B+.
- class `BPlusTree< T >`
Implementación genérica de un árbol B+.

6.2.1. Descripción detallada

Implementación de un árbol B+ genérico.

Este árbol B+ permite insertar, buscar y eliminar elementos de tipo T.

6.3. BPlusTree.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #include <iostream>
00002
00007
00015 template <typename T> struct Node {
00016     bool is_leaf;
00017     int degree;
00018     int size;
00019     T *item;
00020     Node<T> **children;
00021     Node<T> *parent;
00022
00023 public:
00028     Node(int _degree);
00029 };
00030
00042 template <typename T> class BPlusTree {
00043     Node<T> *root;
00044     int degree;
00045
00046 public:
00051     BPlusTree(int _degree);
00052
00056     ~BPlusTree();
00057
00065     Node<T> *_findKey(Node<T> *node, T key);
00066
00073     Node<T> *_findRange(Node<T> *node, T key);
00074
00082     int _findIndex(T *arr, T data, int len);
00083
00092     Node<T> **_innerInsert(Node<T> **child_arr, Node<T> *child, int len,
00093                             int index);
00094
00102     T *_insertItem(T *arr, T data, int len);
00103
00111     Node<T> *_insertInChild(Node<T> *node, T data, Node<T> *child);
00112
00119     void _insertInParent(Node<T> *par, Node<T> *child, T data);
00120
00127     void _removeFromParent(Node<T> *node, int index, Node<T> *par);
00128
00133     void _print(Node<T> *cursor);
```

```

00134
00139     Node<T> *getRoot();
00140
00149     int searchRange(T start, T end, T *result_data, int arr_length);
00150
00156     bool search(T data);
00157
00162     void insert(T data);
00163
00168     void remove(T data);
00169
00174     void clear(Node<T> *cursor);
00175
00179     void print();
00180 };
00181
00189 template <typename T> Node<T>::Node(int _degree) {
00190     this->is_leaf = false;
00191     this->degree = _degree;
00192     this->size = 0;
00193
00194     T *_item = new T[degree - 1]();
00195     this->item = _item;
00196
00197     Node<T> **_children = new Node<T> *[degree];
00198     for (int i = 0; i < degree; i++) {
00199         _children[i] = nullptr;
00200     }
00201     this->children = _children;
00202
00203     this->parent = nullptr;
00204 }
00205
00206 template <typename T> BPlusTree<T>::BPlusTree(int _degree) {
00207     this->root = nullptr;
00208     this->degree = _degree;
00209 }
00210
00211 template <typename T> BPlusTree<T>::~BPlusTree() { clear(this->root); }
00212
00213 template <typename T> Node<T> *BPlusTree<T>::getRoot() { return this->root; }
00214
00215 template <typename T> Node<T> *BPlusTree<T>::_findKey(Node<T> *node, T key) {
00216     if (node == nullptr) {
00217         return nullptr;
00218     } else {
00219         Node<T> *cursor = node;
00220
00221         while (!cursor->is_leaf) {
00222             for (int i = 0; i < cursor->size; i++) {
00223                 if (key < cursor->item[i]) {
00224
00225                     cursor = cursor->children[i];
00226                     break;
00227                 }
00228                 if (i == (cursor->size) - 1) {
00229                     cursor = cursor->children[i + 1];
00230                     break;
00231                 }
00232             }
00233         }
00234
00235         for (int i = 0; i < cursor->size; i++) {
00236             if (cursor->item[i] == key) {
00237                 return cursor;
00238             }
00239         }
00240
00241         return nullptr;
00242     }
00243 }
00244
00245 template <typename T> Node<T> *BPlusTree<T>::_findRange(Node<T> *node, T key) {
00246     if (node == nullptr) {
00247         return nullptr;
00248     } else {
00249         Node<T> *cursor = node;
00250
00251         while (!cursor->is_leaf) {
00252             for (int i = 0; i < cursor->size; i++) {
00253                 if (key < cursor->item[i]) {
00254
00255                     cursor = cursor->children[i];
00256                     break;
00257                 }
00258                 if (i == (cursor->size) - 1) {
00259                     cursor = cursor->children[i + 1];

```

```

00260         break;
00261     }
00262 }
00263 }
00264 return cursor;
00265 }
00266 }
00267
00268 template <typename T>
00269 int BPlusTree<T>::searchRange(T start, T end, T *result_data, int arr_length) {
00270     int index = 0;
00271
00272     Node<T> *start_node = _findRange(this->root, start);
00273     Node<T> *cursor = start_node;
00274     T temp = cursor->item[0];
00275
00276     while (temp <= end) {
00277         if (cursor == nullptr) {
00278             break;
00279         }
00280         for (int i = 0; i < cursor->size; i++) {
00281             temp = cursor->item[i];
00282             if ((temp >= start) && (temp <= end)) {
00283                 result_data[index] = temp;
00284                 index++;
00285             }
00286         }
00287         cursor = cursor->children[cursor->size];
00288     }
00289     return index;
00290 }
00291
00292 template <typename T> bool BPlusTree<T>::search(T data) {
00293     return _findKey(this->root, data) != nullptr;
00294 }
00295
00296 template <typename T> int BPlusTree<T>::_findIndex(T *arr, T data, int len) {
00297     int index = 0;
00298     for (int i = 0; i < len; i++) {
00299         if (data < arr[i]) {
00300             index = i;
00301             break;
00302         }
00303         if (i == len - 1) {
00304             index = len;
00305             break;
00306         }
00307     }
00308     return index;
00309 }
00310
00311 template <typename T> T *BPlusTree<T>::_insertItem(T *arr, T data, int len) {
00312     int index = 0;
00313     for (int i = 0; i < len; i++) {
00314         if (data < arr[i]) {
00315             index = i;
00316             break;
00317         }
00318         if (i == len - 1) {
00319             index = len;
00320             break;
00321         }
00322     }
00323
00324     for (int i = len; i > index; i--) {
00325         arr[i] = arr[i - 1];
00326     }
00327
00328     arr[index] = data;
00329
00330     return arr;
00331 }
00332
00333 template <typename T>
00334 Node<T> **BPlusTree<T>::_innerInsert(Node<T> **child_arr, Node<T> *child,
00335                                     int len, int index) {
00336     for (int i = len; i > index; i--) {
00337         child_arr[i] = child_arr[i - 1];
00338     }
00339     child_arr[index] = child;
00340     return child_arr;
00341 }
00342
00343 template <typename T>
00344 Node<T> *BPlusTree<T>::_insertInChild(Node<T> *node, T data, Node<T> *child) {
00345     int item_index = 0;
00346     int child_index = 0;

```

```

00347     for (int i = 0; i < node->size; i++) {
00348         if (data < node->item[i]) {
00349             item_index = i;
00350             child_index = i + 1;
00351             break;
00352         }
00353         if (i == node->size - 1) {
00354             item_index = node->size;
00355             child_index = node->size + 1;
00356             break;
00357         }
00358     }
00359     for (int i = node->size; i > item_index; i--) {
00360         node->item[i] = node->item[i - 1];
00361     }
00362     for (int i = node->size + 1; i > child_index; i--) {
00363         node->children[i] = node->children[i - 1];
00364     }
00365     node->item[item_index] = data;
00366     node->children[child_index] = child;
00367     return node;
00370 }
00371
00372 template <typename T>
00373 void BPlusTree<T>::_insertInParent(Node<T> *par, Node<T> *child, T data) {
00374
00375     Node<T> *cursor = par;
00376     if (cursor->size < this->degree - 1) {
00377
00378         cursor = _insertInChild(cursor, data, child);
00379         cursor->size++;
00380     } else {
00381
00382         auto *Newnode = new Node<T>(this->degree);
00383         Newnode->parent = cursor->parent;
00384
00385         T *item_copy = new T[cursor->size + 1];
00386         for (int i = 0; i < cursor->size; i++) {
00387             item_copy[i] = cursor->item[i];
00388         }
00389         item_copy = _insertItem(item_copy, data, cursor->size);
00390
00391         auto **child_copy = new Node<T> *[cursor->size + 2];
00392         for (int i = 0; i < cursor->size + 1; i++) {
00393             child_copy[i] = cursor->children[i];
00394         }
00395         child_copy[cursor->size + 1] = nullptr;
00396         child_copy = _innerInsert(child_copy, child, cursor->size + 1,
00397                                 _findIndex(item_copy, data, cursor->size + 1));
00398
00399         cursor->size = (this->degree) / 2;
00400         if ((this->degree) % 2 == 0) {
00401             Newnode->size = (this->degree) / 2 - 1;
00402         } else {
00403             Newnode->size = (this->degree) / 2;
00404         }
00405
00406         for (int i = 0; i < cursor->size; i++) {
00407             cursor->item[i] = item_copy[i];
00408             cursor->children[i] = child_copy[i];
00409         }
00410         cursor->children[cursor->size] = child_copy[cursor->size];
00411
00412         for (int i = 0; i < Newnode->size; i++) {
00413             Newnode->item[i] = item_copy[cursor->size + i + 1];
00414             Newnode->children[i] = child_copy[cursor->size + i + 1];
00415             Newnode->children[i]->parent = Newnode;
00416         }
00417         Newnode->children[Newnode->size] =
00418             child_copy[cursor->size + Newnode->size + 1];
00419         Newnode->children[Newnode->size]->parent = Newnode;
00420
00421         T paritem = item_copy[this->degree / 2];
00422
00423         delete[] item_copy;
00424         delete[] child_copy;
00425
00426         if (cursor->parent == nullptr) {
00427             auto *Newparent = new Node<T>(this->degree);
00428             cursor->parent = Newparent;
00429             Newnode->parent = Newparent;
00430
00431             Newparent->item[0] = paritem;
00432             Newparent->size++;
00433

```

```

00434     Newparent->children[0] = cursor;
00435     Newparent->children[1] = Newnode;
00436
00437     this->root = Newparent;
00438
00439 } else {
00440     _insertInParent(cursor->parent, Newnode, paritem);
00441 }
00442 }
00443 }
00444
00445 template <typename T> void BPlusTree<T>::insert(T data) {
00446     if (this->root == nullptr) {
00447         this->root = new Node<T>(this->degree);
00448         this->root->is_leaf = true;
00449         this->root->item[0] = data;
00450         this->root->size = 1;
00451     } else {
00452         Node<T> *cursor = this->root;
00453
00454         cursor = _findRange(cursor, data);
00455
00456         if (cursor->size < (this->degree - 1)) {
00457
00458             cursor->item = _insertItem(cursor->item, data, cursor->size);
00459             cursor->size++;
00460
00461             cursor->children[cursor->size] = cursor->children[cursor->size - 1];
00462             cursor->children[cursor->size - 1] = nullptr;
00463         } else {
00464
00465             auto *Newnode = new Node<T>(this->degree);
00466             Newnode->is_leaf = true;
00467             Newnode->parent = cursor->parent;
00468
00469             T *item_copy = new T[cursor->size + 1];
00470             for (int i = 0; i < cursor->size; i++) {
00471                 item_copy[i] = cursor->item[i];
00472             }
00473
00474             item_copy = _insertItem(item_copy, data, cursor->size);
00475
00476             cursor->size = (this->degree) / 2;
00477             if ((this->degree) % 2 == 0) {
00478                 Newnode->size = (this->degree) / 2;
00479             } else {
00480                 Newnode->size = (this->degree) / 2 + 1;
00481             }
00482
00483             for (int i = 0; i < cursor->size; i++) {
00484                 cursor->item[i] = item_copy[i];
00485             }
00486             for (int i = 0; i < Newnode->size; i++) {
00487                 Newnode->item[i] = item_copy[cursor->size + i];
00488             }
00489
00490             cursor->children[cursor->size] = Newnode;
00491             Newnode->children[Newnode->size] = cursor->children[this->degree - 1];
00492             cursor->children[this->degree - 1] = nullptr;
00493
00494             delete[] item_copy;
00495
00496             T paritem = Newnode->item[0];
00497
00498             if (cursor->parent == nullptr) {
00499                 auto *Newparent = new Node<T>(this->degree);
00500                 cursor->parent = Newparent;
00501                 Newnode->parent = Newparent;
00502
00503                 Newparent->item[0] = paritem;
00504                 Newparent->size++;
00505
00506                 Newparent->children[0] = cursor;
00507                 Newparent->children[1] = Newnode;
00508
00509                 this->root = Newparent;
00510             } else {
00511                 _insertInParent(cursor->parent, Newnode, paritem);
00512             }
00513         }
00514     }
00515 }
00516
00517 template <typename T> void BPlusTree<T>::remove(T data) {
00518
00519     Node<T> *cursor = this->root;
00520

```

```

00521     cursor = _findRange(cursor, data);
00522
00523     int sib_index = -1;
00524     for (int i = 0; i < cursor->parent->size + 1; i++) {
00525         if (cursor == cursor->parent->children[i]) {
00526             sib_index = i;
00527         }
00528     }
00529     int left = sib_index - 1;
00530     int right = sib_index + 1;
00531
00532     int del_index = -1;
00533     for (int i = 0; i < cursor->size; i++) {
00534         if (cursor->item[i] == data) {
00535             del_index = i;
00536             break;
00537         }
00538     }
00539
00540     if (del_index == -1) {
00541         return;
00542     }
00543
00544     for (int i = del_index; i < cursor->size - 1; i++) {
00545         cursor->item[i] = cursor->item[i + 1];
00546     }
00547     cursor->item[cursor->size - 1] = T{};
00548     cursor->size--;
00549
00550     if (cursor == this->root && cursor->size == 0) {
00551         clear(this->root);
00552         this->root = nullptr;
00553         return;
00554     }
00555     cursor->children[cursor->size] = cursor->children[cursor->size + 1];
00556     cursor->children[cursor->size + 1] = nullptr;
00557
00558     if (cursor == this->root) {
00559         return;
00560     }
00561     if (cursor->size < degree / 2) {
00562
00563         if (left >= 0) {
00564             Node<T> *leftsibling = cursor->parent->children[left];
00565
00566             if (leftsibling->size > degree / 2) {
00567                 T *temp = new T[cursor->size + 1];
00568
00569                 for (int i = 0; i < cursor->size; i++) {
00570                     temp[i] = cursor->item[i];
00571                 }
00572
00573                 _insertItem(temp, leftsibling->item[leftsibling->size - 1],
00574                             cursor->size);
00575                 for (int i = 0; i < cursor->size + 1; i++) {
00576                     cursor->item[i] = temp[i];
00577                 }
00578                 cursor->size++;
00579                 delete[] temp;
00580
00581                 cursor->children[cursor->size] = cursor->children[cursor->size - 1];
00582                 cursor->children[cursor->size - 1] = nullptr;
00583
00584                 leftsibling->item[leftsibling->size - 1] = T{};
00585                 leftsibling->size--;
00586                 leftsibling->children[leftsibling->size] =
00587                     leftsibling->children[leftsibling->size + 1];
00588                 leftsibling->children[leftsibling->size + 1] = nullptr;
00589
00590                 cursor->parent->item[left] = cursor->item[0];
00591
00592                 return;
00593             }
00594         }
00595         if (right <= cursor->parent->size) {
00596             Node<T> *rightsibling = cursor->parent->children[right];
00597
00598             if (rightsibling->size > degree / 2) {
00599                 T *temp = new T[cursor->size + 1];
00600
00601                 for (int i = 0; i < cursor->size; i++) {
00602                     temp[i] = cursor->item[i];
00603                 }
00604
00605                 _insertItem(temp, rightsibling->item[0], cursor->size);
00606                 for (int i = 0; i < cursor->size + 1; i++) {
00607                     cursor->item[i] = temp[i];

```

```

00608     }
00609     cursor->size++;
00610     delete[] temp;
00611
00612     cursor->children[cursor->size] = cursor->children[cursor->size - 1];
00613     cursor->children[cursor->size - 1] = nullptr;
00614
00615     for (int i = 0; i < rightsibling->size - 1; i++) {
00616         rightsibling->item[i] = rightsibling->item[i + 1];
00617     }
00618     rightsibling->item[rightsibling->size - 1] = T{};
00619     rightsibling->size--;
00620     rightsibling->children[rightsibling->size] =
00621         rightsibling->children[rightsibling->size + 1];
00622     rightsibling->children[rightsibling->size + 1] = nullptr;
00623
00624     cursor->parent->item[right - 1] = rightsibling->item[0];
00625
00626     return;
00627 }
00628 }
00629
00630 if (left >= 0) {
00631     Node<T> *leftsibling = cursor->parent->children[left];
00632
00633     for (int i = 0; i < cursor->size; i++) {
00634         leftsibling->item[leftsibling->size + i] = cursor->item[i];
00635     }
00636
00637     leftsibling->children[leftsibling->size] = nullptr;
00638     leftsibling->size = leftsibling->size + cursor->size;
00639     leftsibling->children[leftsibling->size] = cursor->children[cursor->size];
00640
00641     _removeFromParent(cursor, left, cursor->parent);
00642     for (int i = 0; i < cursor->size; i++) {
00643         cursor->item[i] = T{};
00644         cursor->children[i] = nullptr;
00645     }
00646     cursor->children[cursor->size] = nullptr;
00647
00648     delete[] cursor->item;
00649     delete[] cursor->children;
00650     delete cursor;
00651
00652     return;
00653 }
00654 if (right <= cursor->parent->size) {
00655     Node<T> *rightsibling = cursor->parent->children[right];
00656
00657     for (int i = 0; i < rightsibling->size; i++) {
00658         cursor->item[i + cursor->size] = rightsibling->item[i];
00659     }
00660
00661     cursor->children[cursor->size] = nullptr;
00662     cursor->size = rightsibling->size + cursor->size;
00663     cursor->children[cursor->size] =
00664         rightsibling->children[rightsibling->size];
00665
00666     _removeFromParent(rightsibling, right - 1, cursor->parent);
00667
00668     for (int i = 0; i < rightsibling->size; i++) {
00669         rightsibling->item[i] = T{};
00670         rightsibling->children[i] = nullptr;
00671     }
00672     rightsibling->children[rightsibling->size] = nullptr;
00673
00674     delete[] rightsibling->item;
00675     delete[] rightsibling->children;
00676     delete rightsibling;
00677     return;
00678 }
00679 } else {
00680     return;
00681 }
00682 }
00683 }
00684
00685 template <typename T>
00686 void BPlusTree<T>::_removeFromParent(Node<T> *node, int index, Node<T> *par) {
00687     Node<T> *remover = node;
00688     Node<T> *cursor = par;
00689     T target = cursor->item[index];
00690
00691     if (cursor == this->root && cursor->size == 1) {
00692         if (remover == cursor->children[0]) {
00693             delete[] remover->item;
00694             delete[] remover->children;

```



```

00695     delete remover;
00696     this->root = cursor->children[1];
00697     delete[] cursor->item;
00698     delete[] cursor->children;
00699     delete cursor;
00700     return;
00701 }
00702 if (remover == cursor->children[1]) {
00703     delete[] remover->item;
00704     delete[] remover->children;
00705     delete remover;
00706     this->root = cursor->children[0];
00707     delete[] cursor->item;
00708     delete[] cursor->children;
00709     delete cursor;
00710     return;
00711 }
00712 }
00713
00714 for (int i = index; i < cursor->size - 1; i++) {
00715     cursor->item[i] = cursor->item[i + 1];
00716 }
00717 cursor->item[cursor->size - 1] = T{};
00718
00719 int rem_index = -1;
00720 for (int i = 0; i < cursor->size + 1; i++) {
00721     if (cursor->children[i] == remover) {
00722         rem_index = i;
00723     }
00724 }
00725 if (rem_index == -1) {
00726     return;
00727 }
00728 for (int i = rem_index; i < cursor->size; i++) {
00729     cursor->children[i] = cursor->children[i + 1];
00730 }
00731 cursor->children[cursor->size] = nullptr;
00732 cursor->size--;
00733
00734 if (cursor == this->root) {
00735     return;
00736 }
00737 if (cursor->size < degree / 2) {
00738
00739     int sib_index = -1;
00740     for (int i = 0; i < cursor->parent->size + 1; i++) {
00741         if (cursor == cursor->parent->children[i]) {
00742             sib_index = i;
00743         }
00744     }
00745     int left = sib_index - 1;
00746     int right = sib_index + 1;
00747
00748     if (left >= 0) {
00749         Node<T> *leftsibling = cursor->parent->children[left];
00750
00751         if (leftsibling->size > degree / 2) {
00752             T *temp = new T[cursor->size + 1];
00753
00754             for (int i = 0; i < cursor->size; i++) {
00755                 temp[i] = cursor->item[i];
00756             }
00757
00758             _insertItem(temp, cursor->parent->item[left], cursor->size);
00759             for (int i = 0; i < cursor->size + 1; i++) {
00760                 cursor->item[i] = temp[i];
00761             }
00762             cursor->parent->item[left] = leftsibling->item[leftsibling->size - 1];
00763             delete[] temp;
00764
00765             Node<T> **child_temp = new Node<T> *[cursor->size + 2];
00766
00767             for (int i = 0; i < cursor->size + 1; i++) {
00768                 child_temp[i] = cursor->children[i];
00769             }
00770
00771             _innerInsert(child_temp, leftsibling->children[leftsibling->size],
00772                         cursor->size, 0);
00773
00774             for (int i = 0; i < cursor->size + 2; i++) {
00775                 cursor->children[i] = child_temp[i];
00776             }
00777             delete[] child_temp;
00778
00779             cursor->size++;
00780             leftsibling->size--;
00781             return;

```

```

00782     }
00783 }
00784
00785 if (right <= cursor->parent->size) {
00786     Node<T> *rightsibling = cursor->parent->children[right];
00787
00788     if (rightsibling->size > degree / 2) {
00789         T *temp = new T[cursor->size + 1];
00790
00791         for (int i = 0; i < cursor->size; i++) {
00792             temp[i] = cursor->item[i];
00793         }
00794
00795         _insertItem(temp, cursor->parent->item[sib_index], cursor->size);
00796         for (int i = 0; i < cursor->size + 1; i++) {
00797             cursor->item[i] = temp[i];
00798         }
00799         cursor->parent->item[sib_index] = rightsibling->item[0];
00800         delete[] temp;
00801
00802         cursor->children[cursor->size + 1] = rightsibling->children[0];
00803         for (int i = 0; i < rightsibling->size; i++) {
00804             rightsibling->children[i] = rightsibling->children[i + 1];
00805         }
00806         rightsibling->children[rightsibling->size] = nullptr;
00807
00808         cursor->size++;
00809         rightsibling->size--;
00810         return;
00811     }
00812 }
00813
00814 if (left >= 0) {
00815     Node<T> *leftsibling = cursor->parent->children[left];
00816
00817     leftsibling->item[leftsibling->size] = cursor->parent->item[left];
00818
00819     for (int i = 0; i < cursor->size; i++) {
00820         leftsibling->item[leftsibling->size + i + 1] = cursor->item[i];
00821     }
00822     for (int i = 0; i < cursor->size + 1; i++) {
00823         leftsibling->children[leftsibling->size + i + 1] = cursor->children[i];
00824         cursor->children[i]->parent = leftsibling;
00825     }
00826     for (int i = 0; i < cursor->size + 1; i++) {
00827         cursor->children[i] = nullptr;
00828     }
00829     leftsibling->size = leftsibling->size + cursor->size + 1;
00830
00831     _removeFromParent(cursor, left, cursor->parent);
00832     return;
00833 }
00834 if (right <= cursor->parent->size) {
00835     Node<T> *rightsibling = cursor->parent->children[right];
00836
00837     cursor->item[cursor->size] = cursor->parent->item[right - 1];
00838
00839     for (int i = 0; i < rightsibling->size; i++) {
00840         cursor->item[cursor->size + 1 + i] = rightsibling->item[i];
00841     }
00842     for (int i = 0; i < rightsibling->size + 1; i++) {
00843         cursor->children[cursor->size + i + 1] = rightsibling->children[i];
00844         rightsibling->children[i]->parent = rightsibling;
00845     }
00846     for (int i = 0; i < rightsibling->size + 1; i++) {
00847         rightsibling->children[i] = nullptr;
00848     }
00849
00850     rightsibling->size = rightsibling->size + cursor->size + 1;
00851
00852     _removeFromParent(rightsibling, right - 1, cursor->parent);
00853     return;
00854 }
00855 } else {
00856     return;
00857 }
00858 }
00859
00860 template <typename T> void BPlusTree<T>::clear(Node<T> *cursor) {
00861     if (cursor != nullptr) {
00862         if (!cursor->is_leaf) {
00863             for (int i = 0; i <= cursor->size; i++) {
00864                 clear(cursor->children[i]);
00865             }
00866         }
00867         delete[] cursor->item;
00868         delete[] cursor->children;

```

```

00869     delete cursor;
00870 }
00871 }
00872
00873 template <typename T> void BPlusTree<T>::print() { _print(this->root); }
00874
00875 template <typename T> void BPlusTree<T>::_print(Node<T> *cursor) {
00876
00877     if (cursor != NULL) {
00878         for (int i = 0; i < cursor->size; ++i) {
00879             std::cout << cursor->item[i] << " ";
00880         }
00881         std::cout << "\n";
00882
00883         if (!cursor->is_leaf) {
00884             for (int i = 0; i < cursor->size + 1; ++i) {
00885                 _print(cursor->children[i]);
00886             }
00887         }
00888     }
00889 }

```

6.4. Referencia del archivo src/structures/HashMap.hpp

Implementación de un mapa hash genérico con manejo de colisiones y redimensionamiento automático.

```

#include <iostream>
#include <optional>
#include <sstream>
#include <string>

```

Clases

- class `HashNode< K, V >`
Nodo que almacena un par clave-valor en la tabla hash.
- class `HashMap< K, V >`
Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento.

6.4.1. Descripción detallada

Implementación de un mapa hash genérico con manejo de colisiones y redimensionamiento automático.

Este archivo define las clases `HashNode` y `HashMap`, que permiten almacenar pares clave-valor de tipo genérico, con operaciones de inserción, búsqueda, eliminación y utilidades adicionales.

6.5. HashMap.hpp

[Ir a la documentación de este archivo.](#)

```

00001
00010
00011 #include <iostream>
00012 #include <optional>
00013 #include <sstream>
00014 #include <string>
00015
00023 template <typename K, typename V> class HashNode {
00024 public:
00025     V value;

```

```

00026     K key;
00027
00033     HashNode(K key, V value);
00034 };
00035
00048 template <typename K, typename V> class HashMap {
00049     HashNode<K, V> **table;
00050     int capacity;
00051     int size;
00052     long long *pows;
00053
00054 public:
00064     HashMap(int cap);
00065
00069     ~HashMap();
00070
00075     long long *_getPows();
00076
00081     double _getLoadFactor();
00082
00088     std::string _toStringGeneric(const K &val);
00089
00095     int _hashCode(K key);
00096
00103     int _wrap(long long key, int size);
00104
00111     void _insert(K key, V value, HashNode<K, V> **arr);
00112
00116     void _grow();
00117
00123     void insert(K key, V value);
00124
00130     std::optional<V> get(K key);
00131
00137     std::optional<V> remove(K key);
00138
00144     bool containsKey(K key);
00145
00150     int getSize();
00151
00156     bool isEmpty();
00157
00162     HashNode<K, V> **toList();
00163
00167     void display();
00168
00172     void clear();
00173 };
00174
00175 template <typename K, typename V> HashNode<K, V>::HashNode(K key, V value) {
00176     this->value = value;
00177     this->key = key;
00178 }
00179
00180 template <typename K, typename V> HashMap<K, V>::HashMap(int cap) {
00181     capacity = cap;
00182     size = 0;
00183     table = new HashNode<K, V> *[capacity];
00184
00185     for (int i = 0; i < capacity; i++)
00186         table[i] = NULL;
00187
00188     pows = _getPows();
00189 }
00190
00191 template <typename K, typename V> HashMap<K, V>::~~HashMap() {
00192     for (int i = 0; i < capacity; i++) {
00193         if (table[i] != NULL) {
00194             delete table[i];
00195         }
00196     }
00197     delete[] table;
00198 }
00199
00200 template <typename K, typename V> void HashMap<K, V>::_grow() {
00201     HashNode<K, V> **oldTable = toList();
00202     capacity *= 2;
00203     HashNode<K, V> **newTable = new HashNode<K, V> *[capacity];
00204     for (int i = 0; i < capacity; i++) {
00205         HashNode<K, V> curr = *oldTable[i];
00206         _insert(curr.key, curr.value, newTable);
00207     }
00208 }
00209
00210 template <typename K, typename V>
00211 void HashMap<K, V>::_insert(K key, V value, HashNode<K, V> **arr) {
00212     HashNode<K, V> *temp = new HashNode<K, V>(key, value);

```

```

00213
00214     int hash = _hashCode(key);
00215
00216     int i = 0;
00217     while (arr[hash] != NULL && arr[hash]->key != key) {
00218         hash = _wrap(hash + (++i * i + i) / 2, capacity);
00219     }
00220
00221     arr[hash] = temp;
00222 }
00223
00224 template <typename K, typename V> long long *HashMap<K, V>::_getPows() {
00225     long long mod = 10e9 + 7;
00226     long long pow = 1;
00227     long long *pows = new long long[25];
00228     for (int i = 0; i < 25; i++) {
00229         pows[i] = pow;
00230         pow = (pow * 5503) % mod;
00231     }
00232     return pows;
00233 }
00234
00235 template <typename K, typename V>
00236 int HashMap<K, V>::_wrap(long long key, int size) {
00237     key %= size;
00238     if (key < 0)
00239         key += size;
00240     return key;
00241 }
00242
00243 template <typename K, typename V>
00244 std::string HashMap<K, V>::_toStringGeneric(const K &val) {
00245     std::ostringstream oss;
00246     oss << val;
00247     return oss.str();
00248 }
00249
00250 template <typename K, typename V> int HashMap<K, V>::_hashCode(K key) {
00251     long long mod = 10e9 + 9;
00252     long long hash = 0;
00253     long long pow = 1;
00254     int i = 0;
00255     for (char c : _toStringGeneric(key)) {
00256         hash = (hash + c * pow) % mod;
00257         i++;
00258         pow = pows[i];
00259     }
00260     return _wrap(hash, capacity);
00261 }
00262
00263 template <typename K, typename V> double HashMap<K, V>::_getLoadFactor() {
00264     return (double)size / capacity;
00265 }
00266
00267 template <typename K, typename V> void HashMap<K, V>::insert(K key, V value) {
00268     if (_getLoadFactor() > 0.6) {
00269         _grow();
00270     }
00271     _insert(key, value, table);
00272     size++;
00273 }
00274
00275 template <typename K, typename V> bool HashMap<K, V>::containsKey(K key) {
00276
00277     int hash = _hashCode(key);
00278
00279     int i = 0;
00280     while (table[hash] != NULL) {
00281         if (table[hash]->key == key)
00282             return true;
00283         hash = _wrap(hash + (++i * i + i) / 2, capacity);
00284     }
00285
00286     return false;
00287 }
00288
00289 template <typename K, typename V>
00290 std::optional<V> HashMap<K, V>::remove(K key) {
00291     int hash = _hashCode(key);
00292
00293     int i = 0;
00294     while (table[hash] != NULL) {
00295         if (table[hash]->key == key) {
00296             HashNode<K, V> *temp = table[hash];
00297
00298             table[hash] = nullptr;
00299

```

```

00300         size--;
00301         return temp->value;
00302     }
00303     hash = _wrap(hash + (++i * i + i) / 2, capacity);
00304 }
00305 return std::nullopt;
00306 }
00307
00308 template <typename K, typename V> std::optional<V> HashMap<K, V>::get(K key) {
00309     int hashIndex = _hashCode(key);
00310
00311     int i = 0;
00312     while (table[hashIndex] != NULL) {
00313         if (table[hashIndex]->key == key)
00314             return table[hashIndex]->value;
00315         hashIndex = _wrap(hashIndex + (++i * i + i) / 2, capacity);
00316     }
00317     return std::nullopt;
00318 }
00319
00320 template <typename K, typename V> int HashMap<K, V>::getSize() { return size; }
00321
00322 template <typename K, typename V> bool HashMap<K, V>::isEmpty() {
00323     return size == 0;
00324 }
00325
00326 template <typename K, typename V> HashNode<K, V> **HashMap<K, V>::toList() {
00327     HashNode<K, V> **list = new HashNode<K, V> *[size];
00328     int index = 0;
00329     for (int i = 0; i < capacity; i++) {
00330         if (table[i] != NULL) {
00331             list[index++] = table[i];
00332         }
00333     }
00334     return list;
00335 }
00336
00337 template <typename K, typename V> void HashMap<K, V>::display() {
00338     for (int i = 0; i < capacity; i++) {
00339         if (table[i] != NULL)
00340             std::cout << "key = " << table[i]->key << " value = " << table[i]->value
00341                 << std::endl;
00342     }
00343 }
00344
00345 template <typename K, typename V> void HashMap<K, V>::clear() {
00346     for (int i = 0; i < capacity; i++) {
00347         if (table[i] != NULL) {
00348             delete table[i];
00349             table[i] = NULL;
00350         }
00351     }
00352     size = 0;
00353 }

```

6.6. Referencia del archivo src/structures/InvertedIndex.cpp

Implementación de la clase [InvertedIndex](#).

```
#include "InvertedIndex.hpp"
```

6.6.1. Descripción detallada

Implementación de la clase [InvertedIndex](#).

6.7. InvertedIndex.cpp

[Ir a la documentación de este archivo.](#)

```

00001 #include "InvertedIndex.hpp"
00002
00006
00007
00008 InvertedIndex::InvertedIndex(int initialCapacity) : map_(initialCapacity) {}
00009
00010 InvertedIndex::~InvertedIndex() { clear(); }
00011
00012 void InvertedIndex::add(const std::string &key, int rowId) {
00013     if (map_.containsKey(key)) {
00014         std::vector<int> existing = map_.get(key).value();
00015         existing.push_back(rowId);
00016         map_.insert(key, existing);
00017     } else {
00018         std::vector<int> vec;
00019         vec.push_back(rowId);
00020         map_.insert(key, vec);
00021     }
00022 }
00023
00024 std::vector<int> InvertedIndex::get(const std::string &key) {
00025     if (map_.containsKey(key)) {
00026         return map_.get(key).value();
00027     } else {
00028         return {};
00029     }
00030 }
00031
00032 void InvertedIndex::remove(const std::string &key, int rowId) {
00033     if (!map_.containsKey(key))
00034         return;
00035
00036     std::vector<int> existing = map_.get(key).value();
00037
00038     std::vector<int> temp;
00039     for (int x : existing) {
00040         if (x != rowId) {
00041             temp.push_back(x);
00042         }
00043     }
00044     existing = temp;
00045
00046     if (existing.empty()) {
00047         map_.remove(key);
00048     } else {
00049         map_.insert(key, existing);
00050     }
00051 }
00052
00053 void InvertedIndex::clear() { map_.clear(); }

```

6.8. Referencia del archivo src/structures/InvertedIndex.hpp

Implementación de un índice invertido utilizando un mapa hash.

```

#include "HashMap.hpp"
#include <string>
#include <vector>

```

Clases

- class [InvertedIndex](#)

Índice invertido basado en [HashMap](#) para asociar claves con listas de identificadores.

6.8.1. Descripción detallada

Implementación de un índice invertido utilizando un mapa hash.

Esta clase permite asociar claves (palabras, términos, etc.) con listas de identificadores de filas, facilitando búsquedas eficientes de ocurrencias de términos en colecciones de datos.

6.9. InvertedIndex.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00009
00010 #include "HashMap.hpp"
00011 #include <string>
00012 #include <vector>
00013
00022 class InvertedIndex {
00023 public:
00029     InvertedIndex(int initialCapacity = 128);
00030
00034     ~InvertedIndex();
00035
00041     void add(const std::string &key, int rowId);
00042
00049     std::vector<int> get(const std::string &key);
00050
00057     void remove(const std::string &key, int rowId);
00058
00062     void clear();
00063
00064 private:
00065     HashMap<std::string, std::vector<int>>
00066         map_;
00068 };
```

6.10. Referencia del archivo src/system/IndexEntry.cpp

Implementación de la estructura [IndexEntry](#) para índices de tablas.

```
#include "IndexEntry.hpp"
#include <stdexcept>
```

6.10.1. Descripción detallada

Implementación de la estructura [IndexEntry](#) para índices de tablas.

6.11. IndexEntry.cpp

[Ir a la documentación de este archivo.](#)

```
00001 #include "IndexEntry.hpp"
00002 #include <stdexcept>
00003
00008
00009 bool IndexEntry::operator<(const IndexEntry &other) const {
00010     if (key.index() != other.key.index()) {
00011         throw std::runtime_error("IndexEntry: type mismatch");
00012     }
}
```



```

00013     if (std::holds_alternative<int>(key)) {
00014         return std::get<int>(key) < std::get<int>(other.key);
00015     }
00016     if (std::holds_alternative<double>(key)) {
00017         return std::get<double>(key) < std::get<double>(other.key);
00018     }
00019     return std::get<std::string>(key) < std::get<std::string>(other.key);
00020 }
00021
00022 bool IndexEntry::operator>(const IndexEntry &other) const {
00023     return other < *this;
00024 }
00025
00026 bool IndexEntry::operator<=(const IndexEntry &other) const {
00027     return !(*this > other);
00028 }
00029
00030 bool IndexEntry::operator>=(const IndexEntry &other) const {
00031     return !(*this < other);
00032 }
00033
00034 bool IndexEntry::operator==(const IndexEntry &other) const {
00035     return key == other.key && rowId == other.rowId;
00036 }

```

6.12. Referencia del archivo src/system/IndexEntry.hpp

Definición de la estructura `IndexEntry` para índices de tablas.

```

#include <string>
#include <variant>

```

Clases

- struct `IndexEntry`
Entrada de índice para estructuras de índice (B+ o invertido).

typedefs

- using `Cell` = `std::variant<int, double, std::string>`
Representa el valor de una celda, que puede ser int, double o string.

6.12.1. Descripción detallada

Definición de la estructura `IndexEntry` para índices de tablas.

Contiene la clave (valor de celda) y el identificador de fila correspondiente, junto con operadores de comparación.

6.13. IndexEntry.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00008
00009 #include <string>
00010 #include <variant>
00011
00012 using Cell = std::variant<int, double, std::string>;
00013
00021 struct IndexEntry {
00022     Cell key;
00023     int rowId;
00024
00029     IndexEntry() : key(0), rowId(-1) {}
00030
00036     IndexEntry(const Cell &k, int r) : key(k), rowId(r) {}
00037
00042     bool operator<(const IndexEntry &other) const;
00047     bool operator>(const IndexEntry &other) const;
00052     bool operator<=(const IndexEntry &other) const;
00057     bool operator>=(const IndexEntry &other) const;
00062     bool operator==(const IndexEntry &other) const;
00063 };
```

6.14. Referencia del archivo src/system/Parser.cpp

Implementación del analizador SQL simple con sintaxis de pipas.

```
#include "Parser.hpp"
#include <algorithm>
#include <iostream>
#include <stdexcept>
```

6.14.1. Descripción detallada

Implementación del analizador SQL simple con sintaxis de pipas.

6.15. Parser.cpp

[Ir a la documentación de este archivo.](#)

```
00001
00005
00006 #include "Parser.hpp"
00007 #include <algorithm>
00008 #include <iostream>
00009 #include <stdexcept>
00010
00011 Parser::Parser() {}
00012
00013 Parser::~Parser() {}
00014
00015 bool Parser::executeCommand(const std::string &command) {
00016     try {
00017         ParsedCommand cmd = parseCommand(command);
00018
00019         switch (cmd.type) {
00020             case CommandType::CREATE_TABLE:
00021                 return executeCreateTable(cmd);
00022             case CommandType::INSERT:
00023                 return executeInsert(cmd);
00024             case CommandType::SELECT:
00025                 return executeSelect(cmd);
```

```

00026     case CommandType::DELETE:
00027         return executeDelete(cmd);
00028     case CommandType::CREATE_INDEX:
00029         return executeCreateIndex(cmd);
00030     case CommandType::CREATE_INVERTED_INDEX:
00031         return executeCreateInvertedIndex(cmd);
00032     case CommandType::SHOW_SCHEMA:
00033         return executeShowSchema(cmd);
00034     case CommandType::SHOW_TABLES:
00035         return executeShowTables();
00036     default:
00037         std::cout << "Unknown or unsupported command" << std::endl;
00038         return false;
00039     }
00040 } catch (const std::exception &e) {
00041     std::cout << "Error executing command: " << e.what() << std::endl;
00042     return false;
00043 }
00044 }
00045
00046 ParsedCommand Parser::parseCommand(const std::string &command) {
00047     ParsedCommand cmd;
00048     cmd.type = CommandType::UNKNOWN;
00049
00050     std::vector<std::string> parts = split(command, '|');
00051     if (parts.empty()) {
00052         return cmd;
00053     }
00054
00055     std::string firstPart = trim(parts[0]);
00056     std::string upperFirst = toUpper(firstPart);
00057
00058     if (upperFirst.find("CREATE TABLE") == 0) {
00059         cmd.type = CommandType::CREATE_TABLE;
00060
00061         std::vector<std::string> tokens = split(firstPart, ' ');
00062         if (tokens.size() >= 3) {
00063             cmd.tableName = trim(tokens[2]);
00064         }
00065
00066         if (parts.size() > 1) {
00067             std::vector<std::string> colDefs = split(parts[1], ' ');
00068             for (const std::string &colDef : colDefs) {
00069                 std::string trimmedDef = trim(colDef);
00070                 if (!trimmedDef.empty()) {
00071                     cmd.columns.push_back(trimmedDef);
00072                 }
00073             }
00074         }
00075     } else if (upperFirst.find("INSERT") == 0) {
00076         cmd.type = CommandType::INSERT;
00077
00078         std::vector<std::string> tokens = split(firstPart, ' ');
00079         if (tokens.size() >= 2) {
00080             if (toUpper(tokens[1]) == "INTO" && tokens.size() >= 3) {
00081                 cmd.tableName = trim(tokens[2]);
00082             } else {
00083                 cmd.tableName = trim(tokens[1]);
00084             }
00085         }
00086
00087         if (parts.size() > 1) {
00088             std::vector<std::string> values = split(trim(parts[1]), ' ', '\\');
00089
00090             for (const std::string &value : values) {
00091                 std::string trimmedValue = trim(value);
00092                 if (!trimmedValue.empty()) {
00093                     cmd.values.push_back(trimmedValue);
00094                 }
00095             }
00096         }
00097     } else if (upperFirst.find("SELECT") == 0) {
00098         cmd.type = CommandType::SELECT;
00099
00100         std::vector<std::string> tokens = split(firstPart, ' ');
00101         for (size_t i = 0; i < tokens.size(); ++i) {
00102             if (toUpper(tokens[i]) == "FROM" && i + 1 < tokens.size()) {
00103                 cmd.tableName = trim(tokens[i + 1]);
00104                 break;
00105             }
00106         }
00107
00108         if (parts.size() > 1) {
00109             std::string wherePart = trim(parts[1]);
00110             if (toUpper(wherePart).find("WHERE") == 0) {
00111                 std::vector<std::string> whereTokens = split(wherePart, ' ', '\\');
00112                 if (whereTokens.size() >= 4) {

```

```

00113         cmd.whereColumn = trim(whereTokens[1]);
00114         cmd.whereValue = trim(whereTokens[3]);
00115     }
00116 }
00117 }
00118 } else if (upperFirst.find("DELETE") == 0) {
00119     cmd.type = CommandType::DELETE;
00120
00121     std::vector<std::string> tokens = split(firstPart, ' ');
00122     for (size_t i = 0; i < tokens.size(); ++i) {
00123         if (toUpper(tokens[i]) == "FROM" && i + 1 < tokens.size()) {
00124             cmd.tableName = trim(tokens[i + 1]);
00125             break;
00126         }
00127     }
00128
00129     if (parts.size() > 1) {
00130         std::string wherePart = trim(parts[1]);
00131         if (toUpper(wherePart).find("WHERE") == 0) {
00132             std::vector<std::string> whereTokens = split(wherePart, ' ', '\\');
00133             if (whereTokens.size() >= 4) {
00134                 cmd.whereColumn = trim(whereTokens[1]);
00135                 cmd.whereValue = trim(whereTokens[3]);
00136             }
00137         }
00138     }
00139 } else if (upperFirst.find("CREATE INDEX") == 0) {
00140     cmd.type = CommandType::CREATE_INDEX;
00141
00142     std::vector<std::string> tokens = split(firstPart, ' ');
00143     for (size_t i = 0; i < tokens.size(); ++i) {
00144         if (toUpper(tokens[i]) == "ON" && i + 1 < tokens.size()) {
00145             cmd.tableName = trim(tokens[i + 1]);
00146             break;
00147         }
00148     }
00149
00150     if (parts.size() > 1) {
00151         std::vector<std::string> indexTokens = split(trim(parts[1]), ' ');
00152         if (!indexTokens.empty()) {
00153             cmd.indexColumn = trim(indexTokens[0]);
00154             if (indexTokens.size() > 1) {
00155                 try {
00156                     cmd.indexDegree = std::stoi(trim(indexTokens[1]));
00157                 } catch (...) {
00158                     cmd.indexDegree = 3;
00159                 }
00160             }
00161         }
00162     }
00163 } else if (upperFirst.find("CREATE INVERTED INDEX") == 0) {
00164     cmd.type = CommandType::CREATE_INVERTED_INDEX;
00165
00166     std::vector<std::string> tokens = split(firstPart, ' ');
00167     for (size_t i = 0; i < tokens.size(); ++i) {
00168         if (toUpper(tokens[i]) == "ON" && i + 1 < tokens.size()) {
00169             cmd.tableName = trim(tokens[i + 1]);
00170             break;
00171         }
00172     }
00173
00174     if (parts.size() > 1) {
00175         cmd.indexColumn = trim(parts[1]);
00176     }
00177 } else if (upperFirst.find("SHOW SCHEMA") == 0) {
00178     cmd.type = CommandType::SHOW_SCHEMA;
00179     std::vector<std::string> tokens = split(firstPart, ' ');
00180     if (tokens.size() >= 3) {
00181         cmd.tableName = trim(tokens[2]);
00182     }
00183 } else if (upperFirst.find("SHOW TABLES") == 0) {
00184     cmd.type = CommandType::SHOW_TABLES;
00185 }
00186
00187 return cmd;
00188 }
00189
00190 Table *Parser::getTable(const std::string &tableName) {
00191     auto it = tables_.find(tableName);
00192     if (it != tables_.end()) {
00193         return it->second.get();
00194     }
00195     return nullptr;
00196 }
00197
00198 std::vector<std::string> Parser::listTables() const {
00199     std::vector<std::string> tableNames;

```

```

00200     for (const auto &pair : tables_) {
00201         tableNames.push_back(pair.first);
00202     }
00203     return tableNames;
00204 }
00205
00206 bool Parser::executeCreateTable(const ParsedCommand &cmd) {
00207     if (cmd.tableName.empty()) {
00208         std::cout << "Error: Table name is required" << std::endl;
00209         return false;
00210     }
00211
00212     if (tables_.find(cmd.tableName) != tables_.end()) {
00213         std::cout << "Error: Table '" << cmd.tableName << "' already exists"
00214             << std::endl;
00215         return false;
00216     }
00217
00218     std::vector<std::pair<std::string, DataType>> columns;
00219     for (const std::string &colDef : cmd.columns) {
00220         size_t colonPos = colDef.find(':');
00221         if (colonPos == std::string::npos) {
00222             std::cout << "Error: Invalid column definition '" << colDef << "'"
00223                 << std::endl;
00224             return false;
00225         }
00226
00227         std::string colName = trim(colDef.substr(0, colonPos));
00228         std::string colType = trim(colDef.substr(colonPos + 1));
00229
00230         if (colName.empty() || colType.empty()) {
00231             std::cout << "Error: Invalid column definition '" << colDef << "'"
00232                 << std::endl;
00233             return false;
00234         }
00235
00236         DataType dataType = parseDataType(colType);
00237         columns.push_back({colName, dataType});
00238     }
00239
00240     if (columns.empty()) {
00241         std::cout << "Error: At least one column is required" << std::endl;
00242         return false;
00243     }
00244
00245     try {
00246         tables_[cmd.tableName] = std::make_unique<Table>(columns);
00247         std::cout << "Table '" << cmd.tableName << "' created successfully"
00248             << std::endl;
00249         return true;
00250     } catch (const std::exception &e) {
00251         std::cout << "Error creating table: " << e.what() << std::endl;
00252         return false;
00253     }
00254 }
00255
00256 bool Parser::executeInsert(const ParsedCommand &cmd) {
00257     Table *table = getTable(cmd.tableName);
00258     if (!table) {
00259         std::cout << "Error: Table '" << cmd.tableName << "' not found"
00260             << std::endl;
00261         return false;
00262     }
00263
00264     std::vector<Cell> row;
00265     const std::vector<DataType> &schema = table->getSchema();
00266     if (cmd.values.size() != schema.size()) {
00267         std::cout << "Error: Value count does not match table schema" << std::endl;
00268         return false;
00269     }
00270
00271     for (size_t i = 0; i < cmd.values.size(); ++i) {
00272         const std::string &value = cmd.values[i];
00273         DataType type = schema[i];
00274         Cell cell = parseValue(value, type);
00275         row.push_back(cell);
00276     }
00277
00278     try {
00279         table->insertRow(row);
00280         std::cout << "Row inserted successfully" << std::endl;
00281         return true;
00282     } catch (const std::exception &e) {
00283         std::cout << "Error inserting row: " << e.what() << std::endl;
00284         return false;
00285     }
00286 }

```

```

00287
00288 bool Parser::executeSelect(const ParsedCommand &cmd) {
00289     Table *table = getTable(cmd.tableName);
00290     if (!table) {
00291         std::cout << "Error: Table '" << cmd.tableName << "' not found"
00292                 << std::endl;
00293         return false;
00294     }
00295
00296     if (cmd.whereColumn.empty()) {
00297         table->printAllRows();
00298     } else {
00299         try {
00300             DataType columnType = table->getColumnType(cmd.whereColumn);
00301             Cell searchValue = parseValue(cmd.whereValue, columnType);
00302             std::vector<int> rowIndices =
00303                 table->search(cmd.whereColumn, searchValue);
00304
00305             if (rowIndices.empty()) {
00306                 std::cout << "No rows found matching criteria" << std::endl;
00307             } else {
00308                 for (int rowIndex : rowIndices) {
00309                     std::vector<Cell> &row = table->getRow(rowIndex);
00310                     for (const Cell &cell : row) {
00311                         std::visit([&](const auto &value) { std::cout << value << " "; },
00312                                 cell);
00313                     }
00314                     std::cout << std::endl;
00315                 }
00316             }
00317         } catch (const std::exception &e) {
00318             std::cout << "Error executing select: " << e.what() << std::endl;
00319             return false;
00320         }
00321     }
00322     return true;
00323 }
00324
00325 bool Parser::executeDelete(const ParsedCommand &cmd) {
00326     Table *table = getTable(cmd.tableName);
00327     if (!table) {
00328         std::cout << "Error: Table '" << cmd.tableName << "' not found"
00329                 << std::endl;
00330         return false;
00331     }
00332
00333     if (cmd.whereColumn.empty() || cmd.whereValue.empty()) {
00334         std::cout << "Error: DELETE requires WHERE clause" << std::endl;
00335         return false;
00336     }
00337
00338     try {
00339         DataType columnType = table->getColumnType(cmd.whereColumn);
00340         Cell searchValue = parseValue(cmd.whereValue, columnType);
00341         std::vector<int> rowIndices =
00342             table->search(cmd.whereColumn, searchValue);
00343
00344         std::sort(rowIndices.rbegin(), rowIndices.rend());
00345
00346         int deletedCount = 0;
00347         for (int rowIndex : rowIndices) {
00348             table->deleteRow(rowIndex);
00349             deletedCount++;
00350         }
00351
00352         std::cout << "Deleted " << deletedCount << " row(s)" << std::endl;
00353         return true;
00354     } catch (const std::exception &e) {
00355         std::cout << "Error executing delete: " << e.what() << std::endl;
00356         return false;
00357     }
00358 }
00359
00360 bool Parser::executeCreateIndex(const ParsedCommand &cmd) {
00361     Table *table = getTable(cmd.tableName);
00362     if (!table) {
00363         std::cout << "Error: Table '" << cmd.tableName << "' not found"
00364                 << std::endl;
00365         return false;
00366     }
00367
00368     if (cmd.indexColumn.empty()) {
00369         std::cout << "Error: Column name is required for index creation"
00370                 << std::endl;
00371     }

```

```

00374     return false;
00375 }
00376
00377 try {
00378     table->createBPlusTreeIndex(cmd.indexColumn, cmd.indexDegree);
00379     std::cout << "B+ Tree index created on column '" << cmd.indexColumn
00380         << "' with degree " << cmd.indexDegree << std::endl;
00381     return true;
00382 } catch (const std::exception &e) {
00383     std::cout << "Error creating index: " << e.what() << std::endl;
00384     return false;
00385 }
00386 }
00387
00388 bool Parser::executeCreateInvertedIndex(const ParsedCommand &cmd) {
00389     Table *table = getTable(cmd.tableName);
00390     if (!table) {
00391         std::cout << "Error: Table '" << cmd.tableName << "' not found"
00392             << std::endl;
00393         return false;
00394     }
00395
00396     if (cmd.indexColumn.empty()) {
00397         std::cout << "Error: Column name is required for inverted index creation"
00398             << std::endl;
00399         return false;
00400     }
00401
00402     try {
00403         table->createInvertedIndex(cmd.indexColumn);
00404         std::cout << "Inverted index created on column '" << cmd.indexColumn << "'"
00405             << std::endl;
00406         return true;
00407     } catch (const std::exception &e) {
00408         std::cout << "Error creating inverted index: " << e.what() << std::endl;
00409         return false;
00410     }
00411 }
00412
00413 bool Parser::executeShowSchema(const ParsedCommand &cmd) {
00414     Table *table = getTable(cmd.tableName);
00415     if (!table) {
00416         std::cout << "Error: Table '" << cmd.tableName << "' not found"
00417             << std::endl;
00418         return false;
00419     }
00420
00421     table->printSchema();
00422     return true;
00423 }
00424
00425 bool Parser::executeShowTables() {
00426     std::vector<std::string> tableNames = listTables();
00427     if (tableNames.empty()) {
00428         std::cout << "No tables found" << std::endl;
00429     } else {
00430         std::cout << "Tables:" << std::endl;
00431         for (const std::string &name : tableNames) {
00432             std::cout << "  " << name << std::endl;
00433         }
00434     }
00435     return true;
00436 }
00437
00438 DataType Parser::parseDataType(const std::string &typeStr) {
00439     std::string upperType = toUpper(typeStr);
00440     if (upperType == "INT" || upperType == "INTEGER") {
00441         return DataType::INT;
00442     } else if (upperType == "DOUBLE" || upperType == "FLOAT") {
00443         return DataType::DOUBLE;
00444     } else if (upperType == "STRING" || upperType == "TEXT") {
00445         return DataType::STRING;
00446     } else {
00447         throw std::invalid_argument("Unknown data type: " + typeStr);
00448     }
00449 }
00450
00451 Cell Parser::parseValue(const std::string &valueStr, DataType dataType) {
00452     switch (dataType) {
00453     case DataType::INT:
00454         return std::stoi(valueStr);
00455     case DataType::DOUBLE:
00456         return std::stod(valueStr);
00457     case DataType::STRING:
00458     default:
00459         return valueStr;
00460     }
}

```

```

00461 }
00462
00463 std::vector<std::string> Parser::split(const std::string &str, char delimiter,
00464                                     char escapeChar) {
00465     std::vector<std::string> tokens;
00466     std::string token;
00467     bool inEscape = false;
00468
00469     if (escapeChar == '\\0') {
00470         for (size_t i = 0; i < str.size(); ++i) {
00471             char c = str[i];
00472             if (c == delimiter) {
00473                 tokens.push_back(token);
00474                 token.clear();
00475             } else {
00476                 token += c;
00477             }
00478         }
00479         tokens.push_back(token);
00480         return tokens;
00481     }
00482
00483     for (size_t i = 0; i < str.size(); ++i) {
00484         char c = str[i];
00485         if (c == escapeChar) {
00486             inEscape = !inEscape;
00487             continue;
00488         }
00489         if (c == delimiter && !inEscape) {
00490             tokens.push_back(token);
00491             token.clear();
00492         } else {
00493             token += c;
00494         }
00495     }
00496     tokens.push_back(token);
00497     return tokens;
00498 }
00499
00500 std::string Parser::trim(const std::string &str) {
00501     size_t start = str.find_first_not_of(" \t\n\r");
00502     if (start == std::string::npos) {
00503         return "";
00504     }
00505
00506     size_t end = str.find_last_not_of(" \t\n\r");
00507     return str.substr(start, end - start + 1);
00508 }
00509
00510 std::string Parser::toUpper(const std::string &str) {
00511     std::string result = str;
00512     std::transform(result.begin(), result.end(), result.begin(), ::toupper);
00513     return result;
00514 }

```

6.16. Referencia del archivo src/system/Parser.hpp

Analizador SQL simple con sintaxis de pipas para operaciones de base de datos.

```

#include "Table.cpp"
#include <memory>
#include <string>
#include <unordered_map>
#include <vector>

```

Clases

- struct [ParsedCommand](#)
Representa un comando SQL analizado con sus componentes.
- class [Parser](#)
Analizador SQL simple con soporte de sintaxis de tubería.

Enumeraciones

- enum class `CommandType` {
`CREATE_TABLE`, `INSERT`, `SELECT`, `DELETE`,
`CREATE_INDEX`, `CREATE_INVERTED_INDEX`, `SHOW_SCHEMA`, `SHOW_TABLES`,
`UNKNOWN` }

Tipos de comandos SQL soportados por el analizador.

6.16.1. Descripción detallada

Analizador SQL simple con sintaxis de pipas para operaciones de base de datos.

Proporciona un analizador que puede interpretar comandos tipo SQL separados por pipas y ejecutarlos sobre objetos `Table`. Soporta operaciones básicas como CREATE, INSERT, SELECT, DELETE e índices.

6.16.2. Documentación de enumeraciones

6.16.2.1. CommandType

```
enum class CommandType [strong]
```

Tipos de comandos SQL soportados por el analizador.

Valores de enumeraciones

<code>CREATE_TABLE</code>	Crear tabla.
<code>INSERT</code>	Insertar datos.
<code>SELECT</code>	Seleccionar datos.
<code>DELETE</code>	Eliminar datos.
<code>CREATE_INDEX</code>	Crear índice B+.
<code>CREATE_INVERTED_INDEX</code>	Crear índice invertido.
<code>SHOW_SCHEMA</code>	Mostrar esquema de tabla.
<code>SHOW_TABLES</code>	Mostrar todas las tablas.
<code>UNKNOWN</code>	Comando desconocido.

6.17. Parser.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00010
00011 #include "Table.cpp"
00012 #include <memory>
00013 #include <string>
00014 #include <unordered_map>
00015 #include <vector>
00016
00021 enum class CommandType {
00022     CREATE_TABLE,
00023     INSERT,
00024     SELECT,
00025     DELETE,
00026     CREATE_INDEX,
00027     CREATE_INVERTED_INDEX,
00028     SHOW_SCHEMA,
```

```

00029     SHOW_TABLES,
00030     UNKNOWN
00031 };
00032
00033 struct ParsedCommand {
00034     CommandType type;
00035     std::string tableName;
00036     std::vector<std::string> columns;
00037     std::vector<std::string> values;
00038     std::string whereColumn;
00039     std::string whereValue;
00040     std::string indexColumn;
00041     int indexDegree = 3;
00042 };
00043
00044 class Parser {
00045 public:
00046     Parser();
00047     ~Parser();
00048
00049     bool executeCommand(const std::string &command);
00050
00051     ParsedCommand parseCommand(const std::string &command);
00052
00053     Table *getTable(const std::string &tableName);
00054
00055     std::vector<std::string> listTables() const;
00056 private:
00057     std::unordered_map<std::string, std::unique_ptr<Table> > tables_;
00058
00059     bool executeCreateTable(const ParsedCommand &cmd);
00060
00061     bool executeInsert(const ParsedCommand &cmd);
00062
00063     bool executeSelect(const ParsedCommand &cmd);
00064
00065     bool executeDelete(const ParsedCommand &cmd);
00066
00067     bool executeCreateIndex(const ParsedCommand &cmd);
00068
00069     bool executeCreateInvertedIndex(const ParsedCommand &cmd);
00070
00071     bool executeShowSchema(const ParsedCommand &cmd);
00072
00073     bool executeShowTables();
00074
00075     DataType parseDataType(const std::string &typeStr);
00076
00077     Cell parseValue(const std::string &valueStr, DataType dataType);
00078
00079     std::vector<std::string> split(const std::string &str, char delimiter,
00080                                   char escapeChar = '\\0');
00081
00082     std::string trim(const std::string &str);
00083
00084     std::string toUpper(const std::string &str);
00085 };

```

6.18. Referencia del archivo src/system/Table.cpp

Implementación de la clase [Table](#).

```

#include "Table.hpp"
#include <climits>
#include <iostream>
#include <sstream>
#include <stdexcept>

```

6.18.1. Descripción detallada

Implementación de la clase [Table](#).

6.19. Table.cpp

[Ir a la documentación de este archivo.](#)

```

00001 #include "Table.hpp"
00002 #include <climits>
00003 #include <iostream>
00004 #include <sstream>
00005 #include <stdexcept>
00006
00010
00011 Table::Table(const std::vector<std::pair<std::string, DataType>> &cols,
00012             int hashCapacity)
00013     : nameToIndex_(hashCapacity), bptIndices_(hashCapacity),
00014       invIndices_(hashCapacity) {
00015
00016     for (int i = 0; i < cols.size(); ++i) {
00017         const auto &[colName, colType] = cols[i];
00018         if (colName.empty()) {
00019             throw std::invalid_argument("Column name cannot be empty");
00020         }
00021
00022         if (nameToIndex_.containsKey(colName)) {
00023             throw std::invalid_argument("Duplicate column name: " + colName);
00024         }
00025         schema_.push_back({colName, colType});
00026         nameToIndex_.insert(colName, i);
00027     }
00028 }
00029
00030 Table::~Table() {
00031
00032     for (int i = 0; i < bptIndices_.getSize(); ++i) {
00033     }
00034
00035     for (int i = 0; i < schema_.size(); ++i) {
00036         const auto &colName = schema_[i].name;
00037         if (bptIndices_.containsKey(colName)) {
00038             BPlusTreeIndexEntry* treePtr = bptIndices_.get(colName).value();
00039             delete treePtr;
00040         }
00041         if (invIndices_.containsKey(colName)) {
00042             InvertedIndex* idxPtr = invIndices_.get(colName).value();
00043             delete idxPtr;
00044         }
00045     }
00046 }
00047
00048 bool Table::cellMatchesType(const Cell &cell, DataType dt) {
00049     switch (dt) {
00050     case DataType::INT:
00051         return std::holds_alternative<int>(cell);
00052     case DataType::DOUBLE:
00053         return std::holds_alternative<double>(cell);
00054     case DataType::STRING:
00055         return std::holds_alternative<std::string>(cell);
00056     }
00057     return false;
00058 }
00059
00060 std::string Table::dataTypeToString(DataType dt) {
00061     switch (dt) {
00062     case DataType::INT:
00063         return "INT";
00064     case DataType::DOUBLE:
00065         return "DOUBLE";
00066     case DataType::STRING:
00067         return "STRING";
00068     }
00069     return "UNKNOWN";
00070 }
00071
00072 int Table::columnIndex(const std::string &colName) {
00073     if (!nameToIndex_.containsKey(colName)) {
00074         throw std::invalid_argument("Unknown column '" + colName + "'");
00075     }
00076     return nameToIndex_.get(colName).value();
00077 }
00078
00079 void Table::printSchema() const {
00080     std::cout << "Schema:\n";
00081     for (const auto &col : schema_) {
00082         std::cout << " " << col.name << " : " << dataTypeToString(col.type)
00083             << "\n";
00084     }
00085 }

```

```

00086
00087 void Table::printAllRows() const {
00088
00089     for (const auto &col : schema_) {
00090         std::cout << col.name << "\t";
00091     }
00092     std::cout << "\n-----\n";
00093
00094     for (int r = 0; r < data_.size(); ++r) {
00095         const auto &row = data_[r];
00096         for (int c = 0; c < row.size(); ++c) {
00097             std::visit([](auto &&val) { std::cout << val << "\t"; }, row[c]);
00098         }
00099         std::cout << "\n";
00100     }
00101 }
00102
00103 std::vector<Cell> &Table::getRow(int rowIndex) {
00104     if (rowIndex < 0 || rowIndex >= data_.size()) {
00105         throw std::out_of_range("Row index out of range");
00106     }
00107     return data_[rowIndex];
00108 }
00109
00110 std::vector<DataType> Table::getSchema() const {
00111     std::vector<DataType> schemaTypes;
00112     for (const auto &col : schema_) {
00113         schemaTypes.push_back(col.type);
00114     }
00115     return schemaTypes;
00116 }
00117
00118 DataType &Table::getColumnType(const std::string &colName) {
00119     int cidx = columnIndex(colName);
00120     return schema_[cidx].type;
00121 }
00122
00123 std::vector<Cell> &Table::deleteRow(int rowIndex) {
00124     if (rowIndex < 0 || rowIndex >= data_.size()) {
00125         throw std::out_of_range("Row index out of range");
00126     }
00127
00128     std::vector<Cell> &deletedRow = data_[rowIndex];
00129
00130     for (int i = 0; i < schema_.size(); ++i) {
00131         const std::string &colName = schema_[i].name;
00132         if (bptIndices_.containsKey(colName)) {
00133             BPlusTree<IndexEntry> *treePtr = bptIndices_.get(colName).value();
00134             IndexEntry ie{deletedRow[i], rowIndex};
00135             treePtr->remove(ie);
00136         }
00137     }
00138
00139     for (int i = 0; i < schema_.size(); ++i) {
00140         if (schema_[i].type == DataType::STRING) {
00141             const std::string &colName = schema_[i].name;
00142             if (invIndices_.containsKey(colName)) {
00143                 InvertedIndex *invPtr = invIndices_.get(colName).value();
00144                 const std::string &keyStr = std::get<std::string>(deletedRow[i]);
00145                 invPtr->remove(keyStr, rowIndex);
00146             }
00147         }
00148     }
00149
00150     data_.erase(data_.begin() + rowIndex);
00151     return deletedRow;
00152 }
00153
00154 Cell &Table::getCell(int rowIndex, const std::string &colName) {
00155     int cidx = columnIndex(colName);
00156     if (rowIndex >= data_.size()) {
00157         throw std::out_of_range("Row index out of range");
00158     }
00159     return data_[rowIndex][cidx];
00160 }
00161
00162 void Table::insertRow(const std::vector<Cell> &row) {
00163     if (row.size() != schema_.size()) {
00164         throw std::invalid_argument("Row size does not match schema size");
00165     }
00166
00167     for (int i = 0; i < row.size(); ++i) {
00168         if (!cellMatchesType(row[i], schema_[i].type)) {
00169             std::ostringstream oss;
00170             oss << "Type mismatch for column '" << schema_[i].name << "' at index "
00171                 << i;
00172             throw std::invalid_argument(oss.str());

```

```

00173     }
00174 }
00175
00176 int newRowId = data_.size();
00177 data_.push_back(row);
00178
00179 for (int i = 0; i < schema_.size(); ++i) {
00180     const std::string &colName = schema_[i].name;
00181     if (bptIndices_.containsKey(colName)) {
00182         BPlusTree<IndexEntry> *treePtr = bptIndices_.get(colName).value();
00183         IndexEntry ie{row[i], newRowId};
00184         treePtr->insert(ie);
00185     }
00186 }
00187
00188 for (int i = 0; i < schema_.size(); ++i) {
00189     if (schema_[i].type == DataType::STRING) {
00190         const std::string &colName = schema_[i].name;
00191         if (invIndices_.containsKey(colName)) {
00192             InvertedIndex *invPtr = invIndices_.get(colName).value();
00193             const std::string &keyStr = std::get<std::string>(row[i]);
00194             invPtr->add(keyStr, newRowId);
00195         }
00196     }
00197 }
00198 }
00199
00200 void Table::createBPlusTreeIndex(const std::string &colName, int degree) {
00201     int cidx = columnIndex(colName);
00202     DataType dt = schema_[cidx].type;
00203
00204     BPlusTree<IndexEntry> *treePtr = new BPlusTree<IndexEntry>(degree);
00205
00206     for (int rid = 0; rid < data_.size(); ++rid) {
00207         const Cell &cell = data_[rid][cidx];
00208         IndexEntry ie{cell, rid};
00209         treePtr->insert(ie);
00210     }
00211
00212     if (bptIndices_.containsKey(colName)) {
00213         delete bptIndices_.get(colName).value();
00214     }
00215     bptIndices_.insert(colName, treePtr);
00216 }
00217
00218 void Table::createInvertedIndex(const std::string &colName) {
00219     int cidx = columnIndex(colName);
00220     if (schema_[cidx].type != DataType::STRING) {
00221         throw std::invalid_argument("Inverted index only valid on STRING columns");
00222     }
00223
00224     InvertedIndex *invPtr = new InvertedIndex();
00225
00226     for (int rid = 0; rid < data_.size(); ++rid) {
00227         const std::string &val = std::get<std::string>(data_[rid][cidx]);
00228         invPtr->add(val, rid);
00229     }
00230
00231     if (invIndices_.containsKey(colName)) {
00232         delete invIndices_.get(colName).value();
00233     }
00234     invIndices_.insert(colName, invPtr);
00235 }
00236
00237 std::vector<int> Table::searchByIndex(const std::string &colName,
00238                                     const Cell &key) {
00239     int cidx = columnIndex(colName);
00240     DataType dt = schema_[cidx].type;
00241
00242     if (bptIndices_.containsKey(colName)) {
00243         BPlusTree<IndexEntry> *treePtr = bptIndices_.get(colName).value();
00244
00245         int maxPossible = data_.size();
00246         IndexEntry *buffer = new IndexEntry[maxPossible];
00247
00248         IndexEntry start{key, 0};
00249         IndexEntry end{key, INT_MAX};
00250
00251         int foundCount = treePtr->searchRange(start, end, buffer, (int)maxPossible);
00252
00253         std::vector<int> result;
00254         result.reserve(foundCount);
00255         for (int i = 0; i < foundCount; ++i) {
00256             result.push_back(buffer[i].rowId);
00257         }
00258         delete[] buffer;
00259         return result;

```

```

00260     }
00261
00262     if (schema_[cidx].type == DataType::STRING &&
00263         invIndices_.containsKey(colName)) {
00264         InvertedIndex *invPtr = invIndices_.get(colName).value();
00265         const std::string &keyStr = std::get<std::string>(key);
00266         return invPtr->get(keyStr);
00267     }
00268
00269     return {};
00270 }
00271
00272 std::vector<int> Table::search(const std::string &colName, const Cell &key) {
00273     if (bptIndices_.containsKey(colName) ||
00274         (columnIndex(colName) >= 0 &&
00275         schema_[columnIndex(colName)].type == DataType::STRING &&
00276         invIndices_.containsKey(colName))) {
00277         return searchByIndex(colName, key);
00278     }
00279
00280     int cidx = columnIndex(colName);
00281     std::vector<int> result;
00282     for (int i = 0; i < data_.size(); ++i) {
00283         if (data_[i][cidx] == key) {
00284             result.push_back(i);
00285         }
00286     }
00287     return result;
00288 }

```

6.20. Referencia del archivo src/system/Table.hpp

Definición de la clase [Table](#) y estructuras auxiliares para la gestión de tablas de datos.

```

#include "../structures/BPlusTree.hpp"
#include "../structures/InvertedIndex.cpp"
#include "IndexEntry.cpp"
#include <string>
#include <variant>
#include <vector>

```

Clases

- struct [Column](#)
Representa una columna de la tabla, con nombre y tipo de dato.
- class [Table](#)
Representa una tabla de datos con soporte para índices y operaciones básicas.

typedefs

- using [Cell](#) = std::variant<int, double, std::string>

Enumeraciones

- enum class [DataType](#) { [INT](#) , [DOUBLE](#) , [STRING](#) }
Tipos de datos soportados por las columnas de la tabla.

6.20.1. Descripción detallada

Definición de la clase [Table](#) y estructuras auxiliares para la gestión de tablas de datos.

Proporciona una estructura de tabla relacional con soporte para índices B+ y de índice invertido, permitiendo operaciones de inserción, búsqueda e impresión de datos.

6.21. Table.hpp

[Ir a la documentación de este archivo.](#)

```

00001
00010
00011 #include "../structures/BPlusTree.hpp"
00012 #include "../structures/InvertedIndex.cpp"
00013 #include "IndexEntry.cpp"
00014 #include <string>
00015 #include <variant>
00016 #include <vector>
00017
00022 enum class DataType { INT, DOUBLE, STRING };
00023
00028 using Cell = std::variant<int, double, std::string>;
00029
00034 struct Column {
00035     std::string name;
00036     DataType type;
00037 };
00038
00047 class Table {
00048 public:
00056     Table(const std::vector<std::pair<std::string, DataType>> &cols,
00057           int hashCapacity = 128);
00058
00062     ~Table();
00063
00068     void insertRow(const std::vector<Cell> &row);
00069
00075     DataType &getColumnType(const std::string &colName);
00076
00080     void printSchema() const;
00081
00085     void printAllRows() const;
00086
00094     Cell &getCell(int rowIndex, const std::string &colName);
00095
00101     std::vector<Cell> &getRow(int rowIndex);
00102
00107     std::vector<DataType> getSchema() const;
00108
00114     std::vector<Cell> &deleteRow(int rowIndex);
00115
00121     void createBPlusTreeIndex(const std::string &colName, int degree);
00122
00127     void createInvertedIndex(const std::string &colName);
00128
00135     std::vector<int> searchByIndex(const std::string &colName, const Cell &key);
00136
00144     std::vector<int> search(const std::string &colName, const Cell &key);
00145
00146 private:
00147     std::vector<Column> schema_;
00148     HashMap<std::string, int>
00149         nameToIndex_;
00150     std::vector<std::vector<Cell>> data_;
00151
00152     HashMap<std::string, BPlusTree<IndexEntry>*>
00153         bptIndices_;
00154     HashMap<std::string, InvertedIndex*>
00155         invIndices_;
00156
00163     static bool cellMatchesType(const Cell &cell, DataType dt);
00164
00170     static std::string dataTypeToString(DataType dt);
00171
00177     int columnIndex(const std::string &colName);
00178 };

```


Índice alfabético

- [_findIndex](#)
BPlusTree< T >, [11](#)
 - [_findKey](#)
BPlusTree< T >, [11](#)
 - [_findRange](#)
BPlusTree< T >, [11](#)
 - [_getLoadFactor](#)
HashMap< K, V >, [19](#)
 - [_getPows](#)
HashMap< K, V >, [19](#)
 - [_hashCode](#)
HashMap< K, V >, [19](#)
 - [_innerInsert](#)
BPlusTree< T >, [12](#)
 - [_insert](#)
HashMap< K, V >, [20](#)
 - [_insertInChild](#)
BPlusTree< T >, [12](#)
 - [_insertInParent](#)
BPlusTree< T >, [12](#)
 - [_insertItem](#)
BPlusTree< T >, [13](#)
 - [_print](#)
BPlusTree< T >, [13](#)
 - [_removeFromParent](#)
BPlusTree< T >, [13](#)
 - [_toStringGeneric](#)
HashMap< K, V >, [20](#)
 - [_wrap](#)
HashMap< K, V >, [20](#)
- [~BPlusTree](#)
BPlusTree< T >, [10](#)
- [~HashMap](#)
HashMap< K, V >, [19](#)
- [~InvertedIndex](#)
InvertedIndex, [27](#)
- [~Table](#)
Table, [38](#)
- [add](#)
InvertedIndex, [27](#)
- [BPlusTree](#)
BPlusTree< T >, [10](#)
- [BPlusTree< T >](#), [9](#)
 - [_findIndex](#), [11](#)
 - [_findKey](#), [11](#)
 - [_findRange](#), [11](#)
 - [_innerInsert](#), [12](#)
 - [_insertInChild](#), [12](#)
 - [_insertInParent](#), [12](#)
 - [_insertItem](#), [13](#)
 - [_print](#), [13](#)
 - [_removeFromParent](#), [13](#)
 - [_toStringGeneric](#), [20](#)
 - [_wrap](#), [20](#)
- [cellMatchesType](#)
Table, [38](#)
- [clear](#)
BPlusTree< T >, [15](#)
- [Column](#), [17](#)
- [columnIndex](#)
Table, [39](#)
- [CommandType](#)
Parser.hpp, [67](#)
- [containsKey](#)
HashMap< K, V >, [21](#)
- [CREATE_INDEX](#)
Parser.hpp, [67](#)
- [CREATE_INVERTED_INDEX](#)
Parser.hpp, [67](#)
- [CREATE_TABLE](#)
Parser.hpp, [67](#)
- [createBPlusTreeIndex](#)
Table, [39](#)
- [createInvertedIndex](#)
Table, [39](#)
- [dataTypeToString](#)
Table, [39](#)
- [DBGS - PSIS](#), [1](#)
- [DELETE](#)
Parser.hpp, [67](#)
- [deleteRow](#)
Table, [40](#)
- [executeCommand](#)
Parser, [31](#)
- [executeCreateIndex](#)
Parser, [31](#)
- [executeCreateInvertedIndex](#)
Parser, [32](#)
- [executeCreateTable](#)

- Parser, 32
- executeDelete
 - Parser, 32
- executeInsert
 - Parser, 33
- executeSelect
 - Parser, 33
- executeShowSchema
 - Parser, 33
- executeShowTables
 - Parser, 34
- get
 - HashMap< K, V >, 21
 - InvertedIndex, 27
- getCell
 - Table, 40
- getColumnType
 - Table, 40
- getRoot
 - BPlusTree< T >, 15
- getRow
 - Table, 41
- getSchema
 - Table, 41
- getSize
 - HashMap< K, V >, 21
- getTable
 - Parser, 34
- HashMap
 - HashMap< K, V >, 18
- HashMap< K, V >, 17
 - _getLoadFactor, 19
 - _getPows, 19
 - _hashCode, 19
 - _insert, 20
 - _toStringGeneric, 20
 - _wrap, 20
 - ~HashMap, 19
 - containsKey, 21
 - get, 21
 - getSize, 21
 - HashMap, 18
 - insert, 21
 - isEmpty, 22
 - remove, 22
 - toList, 22
- HashNode
 - HashNode< K, V >, 23
- HashNode< K, V >, 23
 - HashNode, 23
- IndexEntry, 24
 - IndexEntry, 24
 - operator<, 25
 - operator<=, 25
 - operator>, 25
 - operator>=, 25
- operator==, 25
- INSERT
 - Parser.hpp, 67
- insert
 - BPlusTree< T >, 15
 - HashMap< K, V >, 21
- insertRow
 - Table, 41
- Instalación y Desarrollo, 3
- InvertedIndex, 26
 - ~InvertedIndex, 27
 - add, 27
 - get, 27
 - InvertedIndex, 26
 - remove, 27
- isEmpty
 - HashMap< K, V >, 22
- listTables
 - Parser, 34
- main
 - Main.cpp, 43
- Main.cpp
 - main, 43
- Node
 - Node< T >, 29
- Node< T >, 28
 - Node, 29
- operator<
 - IndexEntry, 25
- operator<=
 - IndexEntry, 25
- operator>
 - IndexEntry, 25
- operator>=
 - IndexEntry, 25
- operator==
 - IndexEntry, 25
- parseCommand
 - Parser, 34
- parseDataType
 - Parser, 35
- ParsedCommand, 29
 - Parser, 30
 - executeCommand, 31
 - executeCreateIndex, 31
 - executeCreateInvertedIndex, 32
 - executeCreateTable, 32
 - executeDelete, 32
 - executeInsert, 33
 - executeSelect, 33
 - executeShowSchema, 33
 - executeShowTables, 34
 - getTable, 34
 - listTables, 34

- parseCommand, [34](#)
 - parseDataType, [35](#)
 - parseValue, [35](#)
 - split, [35](#)
 - toUpper, [36](#)
 - trim, [36](#)
- Parser.hpp
 - CommandType, [67](#)
 - CREATE_INDEX, [67](#)
 - CREATE_INVERTED_INDEX, [67](#)
 - CREATE_TABLE, [67](#)
 - DELETE, [67](#)
 - INSERT, [67](#)
 - SELECT, [67](#)
 - SHOW_SCHEMA, [67](#)
 - SHOW_TABLES, [67](#)
 - UNKNOWN, [67](#)
- parseValue
 - Parser, [35](#)
- remove
 - BPlusTree< T >, [15](#)
 - HashMap< K, V >, [22](#)
 - InvertedIndex, [27](#)
- search
 - BPlusTree< T >, [16](#)
 - Table, [41](#)
- searchByIndex
 - Table, [42](#)
- searchRange
 - BPlusTree< T >, [16](#)
- SELECT
 - Parser.hpp, [67](#)
- SHOW_SCHEMA
 - Parser.hpp, [67](#)
- SHOW_TABLES
 - Parser.hpp, [67](#)
- split
 - Parser, [35](#)
- src/Main.cpp, [43](#)
- src/structures/BPlusTree.hpp, [44](#)
- src/structures/HashMap.hpp, [53](#)
- src/structures/InvertedIndex.cpp, [56](#), [57](#)
- src/structures/InvertedIndex.hpp, [57](#), [58](#)
- src/system/IndexEntry.cpp, [58](#)
- src/system/IndexEntry.hpp, [59](#), [60](#)
- src/system/Parser.cpp, [60](#)
- src/system/Parser.hpp, [66](#), [67](#)
- src/system/Table.cpp, [68](#), [69](#)
- src/system/Table.hpp, [72](#), [73](#)
- Table, [37](#)
 - ~Table, [38](#)
 - cellMatchesType, [38](#)
 - columnIndex, [39](#)
 - createBPlusTreeIndex, [39](#)
 - createInvertedIndex, [39](#)
 - dataTypeToString, [39](#)
 - deleteRow, [40](#)
 - getCell, [40](#)
 - getColumnType, [40](#)
 - getRow, [41](#)
 - getSchema, [41](#)
 - insertRow, [41](#)
 - search, [41](#)
 - searchByIndex, [42](#)
 - Table, [38](#)
 - toList
 - HashMap< K, V >, [22](#)
 - toUpper
 - Parser, [36](#)
 - trim
 - Parser, [36](#)
- UNKNOWN
 - Parser.hpp, [67](#)